Department of Engineering Science

University of Oxford

4th Year Project Report

Closed-loop Deep Brain Stimulation Using Machine Learning for Treatment of Parkinson's Disease

Joshua Ampomah

Keble College

Supervisor: Prof. Mark Cannon

Project Number: 13465

May 2026

Abstract

Implanted deep brain stimulation (DBS) devices treat Parkinson's disease with fixed stimulation. This wastes energy and causes side effects during mild periods. Model predictive control (MPC) could adapt stimulation in real time to a neural biomarker. However, running a learned nonlinear predictor on implant-grade chips has been out of reach. This project builds and compares six closed-loop DBS controllers. These range from reactive baselines to neural-network MPC and a new Koopman MPC. The results show that learned MPC can run on a small chip while beating all simpler options. The main contribution is *stable-neuron elimination*, an exact scheme that removes 96% of the optimisation variables from input-convex neural network programs. Paired with a Koopman predictor that cuts the iterative solve to a single quadratic program, this brings MPC step time to 9.9 ms on an STM32L476RG Cortex-M4F. A 50-trial test uses $\pm 40\%$ parameter mismatch across four patients. Here, Koopman MPC cuts tracking error by 56% and stimulation by 12% against the clinical bang-bang baseline, with passive robustness to parameter drift.

Acknowledgements

I thank my supervisor, Prof. Mark Cannon, for his guidance throughout this project. His insights into model predictive control were essential.

I am grateful to the Brain Network Dynamics Unit at the John Radcliffe Hospital for access to patient data.

I also thank Sebastian Steffen for sharing context on prior work, without which this project would not have been possible.

Contents

Abstract i

Acknowledgements ii

1 Introduction 1

1.1 Report Structure . . . 3

2 Literature Review 4

2.1 Parkinson's Disease and Deep Brain Stimulation . . . 4

2.2 Beta Oscillations as a Biomarker . . . 5

2.3 Closed-Loop Control Strategies . . . 6

2.4 Deep Learning for Neural Prediction . . . 6

2.4.1 Input-Convex Neural Networks . . . 6

2.4.2 Difference-of-Convex Decomposition . . . 6

2.4.3 DCNN-MPC for Closed-Loop DBS . . . 7

2.4.4 Koopman Operator Theory . . . 7

3 System Identification and Modelling 8

3.1 Beta Oscillation Envelope Model . . . 8

3.1.1 Log Transform . . . 9

3.2 Stimulation Response Model . . . 10

3.3 Linearity Analysis . . . 10

3.3.1 Pipeline-Induced Linearity . . . 10

3.3.2 Residual Nonlinearity in Free Dynamics . . . 11

3.4 Shared Setup for Data-Driven Models . . . 11

3.4.1 State Vector . . . 12
3.4.2 Training Data . . . 12
3.5 Multi-step ARX Baseline . . . 12
3.6 Multi-step DCNN Predictor . . . 13
3.6.1 DCNN Architecture . . . 13
3.6.2 Residual-DCNN Variant . . . 14
3.6.3 Network Training . . . 14
3.6.4 Predictor Validation . . . 14
3.7 Koopman Predictor . . . 14
3.7.1 Model Architecture . . . 15

4 Controller Design **16**
4.1 Bang-Bang Controller . . . 16
4.2 Proportional-Integral Control . . . 16
4.3 Multi-step ARX MPC . . . 17
4.4 DCNN Tube MPC . . . 17
4.4.1 DC Decomposition and Tube Bounds . . . 17
4.4.2 QP Subproblem . . . 19
4.4.3 SCP Algorithm . . . 19
4.4.4 Residual-DCNN Tube MPC . . . 20
4.5 Koopman MPC . . . 20

5 Implementation **21**
5.1 From Prototyping to Real-Time . . . 21
5.2 Stable-Neuron Elimination . . . 22
5.2.1 Neuron Classification via Interval Arithmetic . . . 22
5.2.2 Symbolic Expression Propagation . . . 24
5.3 Embedded C++ Implementation . . . 24
5.3.1 Interior-Point Solver . . . 25
5.3.2 Memory Architecture . . . 28
5.3.3 Hardware-in-the-Loop Validation . . . 28
5.3.4 MPC Formulation Comparison . . . 29

6 Evaluation Methodology **31**

6.1 Simulation Framework . . . 31

6.1.1 Patient Data . . . 31

6.1.2 Closed-Loop Simulation . . . 31

6.1.3 Performance Metrics . . . 33

6.2 Online Disturbance Bounds . . . 33

6.2.1 Dvoretzky–Kiefer–Wolfowitz (DKW) Bounds . . . 33

6.2.2 Probably Approximately Correct (PAC) Concentration Bounds . . . 34

6.2.3 Adaptive Conformal Inference (ACI) . . . 34

6.2.4 Method Comparison . . . 35

6.3 Controller Tuning . . . 35

7 Results and Discussion **36**

7.1 50-Trial Controller Comparison . . . 36

7.1.1 Error-Stimulation Trade-off . . . 36

7.1.2 Cost-Optimal Operating Points . . . 37

7.1.3 Representative Traces . . . 38

7.2 Online Bound Behaviour . . . 38

7.3 Drift Robustness Analysis . . . 39

7.3.1 Drift Scenarios . . . 39

7.3.2 Prediction Under Signal Drift . . . 39

7.3.3 Paired Parameter Mismatch . . . 41

7.4 Cross-Patient Evaluation . . . 41

7.4.1 Pareto Frontiers Across Patients . . . 44

7.4.2 Why Rankings Differ . . . 44

8 Conclusions **45**

8.1 Summary . . . 45

8.2 Contributions . . . 46

8.3 Limitations . . . 46

8.4 Future Work . . . 47

Chapter 1

Introduction

Parkinson's disease is the second most common brain disorder. It affects over six million people worldwide, and that number is set to double by 2040 [1]. When drugs alone can no longer control motor symptoms, Deep Brain Stimulation (DBS) is one of the best options. Implanted electrodes send pulses to targeted brain regions, and over 160,000 patients have been treated [2].

Nearly all implanted DBS systems run open-loop, sending constant stimulation no matter how the symptoms change. This wastes battery life, forcing repeated surgery to swap the device, and causes needless side effects during mild periods. A closed-loop system—one that measures a neural marker of symptom severity and adjusts stimulation in real time—solves both problems. Recent trials show the clinical promise of this approach [3, 4]. Yet the only commercial adaptive system still relies on threshold switching, with no proportional modulation or predictive horizon.

Threshold (bang-bang) switching—turning stimulation on or off when a biomarker crosses a set point—is the deployed method. Clinical trials show that even this coarse scheme cuts side effects [3]. Proportional and proportional-integral (PI) controllers go further by grading stimulation amplitude smoothly. However, all of these reactive controllers have no predictive horizon. They also cannot enforce safety and efficiency limits on future stimulation in any planned way. Model predictive control (MPC) fixes both gaps. It uses a dynamic model of the brain's response to plan future inputs, shaping a cost function while respecting constraints at every step. With a linear model, MPC reduces to a convex quadratic programme (QP) that embedded hardware can solve reliably. The snag is that the brain's response is widely held to be nonlinear, time-varying, and patient-specific. A single linear model may then only capture dynamics near one oper-

ating point, and prediction error can grow as conditions drift. Two ideas address this together. *Machine learning* captures the complex input–output map of stimulated neural tissue across a wide operating range. *Robust* MPC then guarantees constraint satisfaction despite residual model error.

This project builds on Steffen and Cannon [5], who placed a difference-of-convex neural network (DCNN) predictor inside a tube MPC controller for closed-loop DBS. Their DCNN tube-MPC setup is the starting point. This project analyses, extends, and implements it end to end. The main contribution is:

- **Stable-neuron elimination for ICNN-based MPC.** A variable-elimination scheme that uses interval arithmetic over the bounded control input set. Each ReLU neuron is classified as stable (fixed activation phase) or unstable, and stable neurons are then removed from the QP. The reduction is exact and cuts the QP from ∼670 to 20–26 variables (∼96%). It applies to any optimisation over an input-convex neural network with bounded decision variables—not only MPC or DBS (Section 5.2).

The rest of the project builds a complete system around this core, adapting known techniques to the DBS domain:

1. **Embedded deployment.** A custom float32 Mehrotra predictor-corrector IPM in C++ runs the neuron-reduced QP on STM32L476RG hardware (80 MHz Cortex-M4F, 128 KB SRAM). It achieves 42.6 ms mean step time with 99.97% convergence (Section 5.3).

2. **Koopman MPC for DBS.** The Koopman MPC framework of Korda and Mezić [6] is adapted to Parkinson's DBS using LASSO-selected analytical lifting features with a linear output map. This keeps the QP convex. It replaces the sequential convex programming (SCP) loop with a single QP per timestep, cutting mean step time to 9.9 ms (Sections 3.7, 4.5 and 5.3.4).

3. **Online disturbance bounds and drift analysis.** Three online bound-estimation methods are compared against the fixed offline bounds of [5]. All three tighten the tube as prediction error shrinks and widen it when it grows. A separate drift study finds that the predictor's autoregressive structure absorbs patient drift passively (Sections 6.2 and 7.3).

4. **Paired 50-trial evaluation.** A head-to-head test of six controllers under matched random parameter mismatch (±40%). All pairwise differences are significant at $p < 10^{-4}$ (Section 7.1).

1.1 Report Structure

Chapter 2 reviews clinical background and prior work. Chapter 3 develops the plant model and predictors. Chapter 4 sets out the controller designs. Chapter 5 describes the stable-neuron QP reduction and embedded deployment. Chapter 6 covers the simulation framework, online disturbance bounds, and test protocol. Chapter 7 gives comparative results and drift robustness analysis. Chapter 8 sums up contributions and future directions.

Chapter 2

Literature Review

This chapter reviews five topics: the case for closed-loop deep brain stimulation, the neural biomarker that drives it, and the shift from threshold switching to model predictive control. It then covers input-convex neural networks and Koopman models.

2.1 Parkinson's Disease and Deep Brain Stimulation

Parkinson's disease (PD) is a progressive brain disorder. Loss of dopamine-making neurons in the substantia nigra, a small midbrain region, disrupts motor circuits. This yields four key motor signs: bradykinesia (slowness), rigidity, resting tremor, and postural instability [7]. PD affects 1–2% of adults over 60. The first-line drug, levodopa, is a dopamine precursor taken orally. Over time it causes motor swings—bouts of good and poor symptom control that shift without warning—and dyskinesias (unwanted movements) in most patients [7].

When drugs alone fail, clinicians add deep brain stimulation (DBS). Electrodes in the subthalamic nucleus (STN, Figure 2.1) send high-rate pulses that suppress abnormal neural firing [8], with pulse levels set once and held fixed regardless of symptom state.

Most DBS systems run open-loop, with three key limits. First, the pulse generator drains its battery in about 4 years and needs surgical replacement [2]. Second, steady pulses cause side effects including tingling, slurred speech, and gait problems [8], and over-stimulate during low-symptom periods [3]. Third, symptom severity varies with activity, drug cycles, and daily rhythms, yet fixed pulses cannot follow these changes [9].

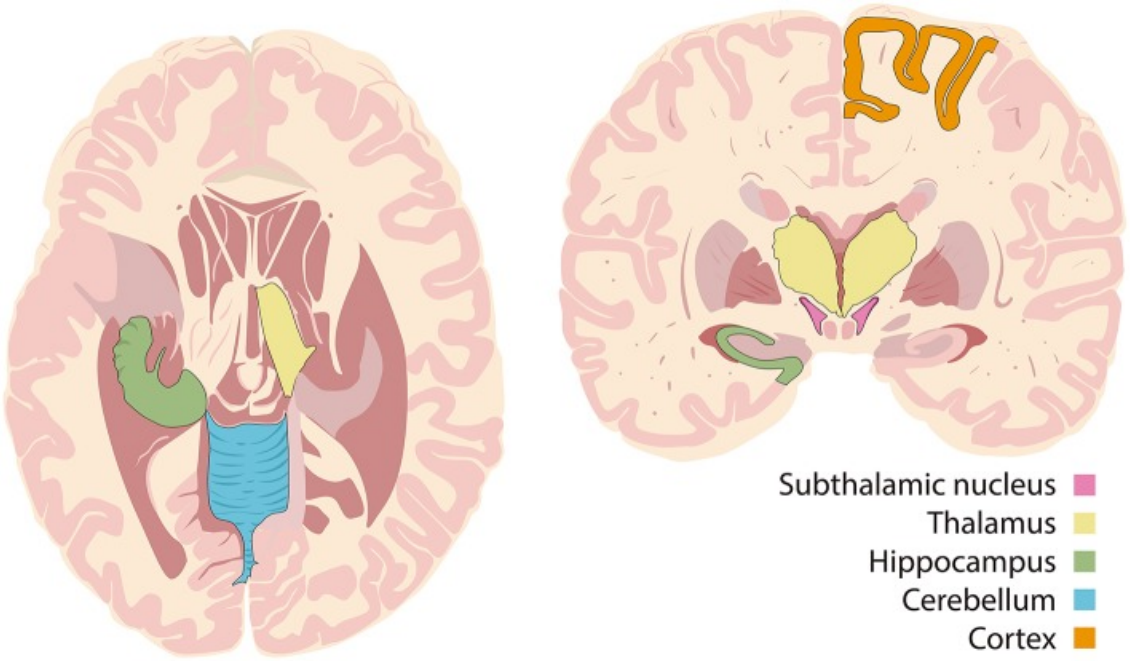

Figure 2.1: Coronal brain sections showing the subthalamic nucleus (STN) and surrounding structures. Adapted from Wikimedia Commons (CC BY 4.0).

In February 2025, the FDA approved the Medtronic Percept PC for BrainSense adaptive stimulation [10]. It tunes pulses in real time via a per-patient biomarker threshold [4]. It is the only adaptive mode on the market as of 2026, and uses simple threshold switching with no predictive element. This gap drives the need for closed-loop methods that predict symptom shifts and scale pulses to match.

2.2 Beta Oscillations as a Biomarker

Of all proposed biomarkers for closed-loop DBS, beta-band oscillations (13–30 Hz) in the STN local field potential are the most widely used [3]. Kühn et al. [11] showed that levodopa cuts STN 8–35 Hz power in step with akinesia-rigidity scores ($r = 0.835$, $P < 0.001$), linking beta suppression to motor benefit. Sustained beta bursts track symptoms more tightly than time-averaged power, and DBS or drugs that suppress them mirror clinical gains [9]. In practice, bandpass filtering and envelope detection extract the beta signal. The aim is to hold it below a per-patient threshold, set to the 75th percentile of beta power recorded during a pre-stimulation baseline window [3].

2.3 Closed-Loop Control Strategies

Threshold controllers turn pulses on when beta power exceeds a set point. This cuts total pulse time but remains reactive and binary [3]. PI control offers smooth changes, yet still reacts only to the current error and must obey rate limits [12]. Model predictive control (MPC) plans future inputs over a receding horizon while enforcing limits on pulse amplitude and its rate of change. Steffen et al. [13] showed that linear MPC beats on-off and PI controllers for DBS, using an augmented state fitted during a pulse-free window. Whether richer state maps and multi-step prediction can do better is a key question for this work.

2.4 Deep Learning for Neural Prediction

The neural dynamics behind DBS are widely held to be nonlinear. Yet most DBS groups choose linear models (ARX, state-space, PI) for tractability, not because linearity holds [12, 13]. Steffen and Cannon [5] tested a nonlinear predictive controller, showing that a DCNN predictor can beat linear AR models on patient data. Whether this edge comes from true nonlinearity at the beta-envelope time scale, or from other factors such as multi-step prediction and richer state maps, remains open.

2.4.1 Input-Convex Neural Networks

When the control input enters the predictor nonlinearly—as in a typical neural net—the MPC problem is non-convex. Amos et al. [14] introduced input-convex neural networks (ICNNs). Three rules—non-negative hidden-to-hidden weights, convex non-decreasing activations (e.g., ReLU), and free passthrough links from the input to each hidden layer—make the output convex in the inputs.

2.4.2 Difference-of-Convex Decomposition

Any continuous function on a compact set equals the gap between two convex functions. So a pair of ICNNs $f = f_1 - f_2$ gives a universal approximator that stays tractable under sequential convex programming. At each step the controller solves a sequential convex program (SCP). Doff-Sotta and Cannon [15] built this DC setup for robust MPC. Lishkova and Cannon [16] proved that the SCP iterates converge for the nominal case and that the robust closed-loop scheme is recursively feasible and stable.

2.4.3 DCNN-MPC for Closed-Loop DBS

Steffen and Cannon [5] used this DC setup for closed-loop DBS with a multi-step DCNN predictor. Each step $i = 1, \dots, N$ has its own ICNN pair $(f_{i,1}, f_{i,2})$. It maps the state z_k and the input sequence $u_{k:k+i-1}$ to the output y_{k+i}:

$$\hat{y}_{k+i} = f_{i,1}(z_k, u_{k:k+i-1}) - f_{i,2}(z_k, u_{k:k+i-1}) + w_i, \quad w_i \in W_i$$

where z_k holds past outputs and inputs, and $W_i = [w_{i,\min}, w_{i,\max}]$ is a bounded error set.

Each net trains on the i-step task alone, avoiding recursive error build-up, with error bounds that depend on Jacobians at the current point and offline disturbance sets. DCNN-MPC cuts both tracking error and control effort by over 20% versus prior methods. A model pretrained on three patients even beats the linear MPC of [13] trained on the test patient's data [5]. Yet SCP solves a chain of QPs at every control step, each adding error that the trust region must contain. A predictor natively linear in the control input would collapse this to a single QP.

2.4.4 Koopman Operator Theory

The Koopman operator gives exactly this structure: an infinite-dimensional linear operator that governs the evolution of observables on a nonlinear state space. Korda and Mezić [6] showed how to approximate it in a finite-dimensional lifted space and pair the lifting with MPC. If the state is mapped by a nonlinear lifting function ψ and the control input enters the lifted dynamics linearly, the predictor is affine in u and the MPC problem is a single QP.

Liang et al. [17] applied Koopman-based MPC to closed-loop neurostimulation in epilepsy, using a deep autoencoder to learn the lifting. Since the dynamics are linear in the lifted state, their MPC problem is a single QP. But the nonlinear decoder means predicted outputs are not linear in the decision variables, so the controller optimises in the lifted space only. Replacing the autoencoder with a linear output map would keep the predicted output linear in u, and thus usable directly in the QP cost.

In sum, DCNN-MPC shows that nonlinear prediction helps closed-loop DBS, but needs a multi-network DC design solved by iterative SCP. A Koopman predictor that captures the same nonlinearity while cutting the solve to one QP would be simpler and faster. This report asks whether such a model works for DBS.

Chapter 3

System Identification and Modelling

This chapter develops the models that underpin control design (Figure 3.1). Two layers are needed. First, a physiological plant model: the beta envelope and its response to stimulation, which together define the closed-loop simulation used to evaluate every controller. Second, data-driven prediction models that MPC rolls forward online. Three variants are developed on a shared state vector. The Multi-step ARX baseline fits one linear regressor per horizon step, and the DCNN swaps this linear map for a learned nonlinear model. A Koopman model then lifts the state with nonlinear features to catch leftover dynamics while keeping outputs affine in control.

3.1 Beta Oscillation Envelope Model

Per Steffen et al. [13], we model the beta envelope $z(t)$ as

$$z(t) = z_\beta(t) \cdot e^{-\eta(t)} + \zeta(t)$$

where $z_\beta(t)$ is the base beta level, $\eta(t) \geq 0$ is the DBS effect, and $\zeta(t)$ is noise. The product form means DBS damps a *share* of beta power: $e^{-\eta(t)} \in [0, 1]$.

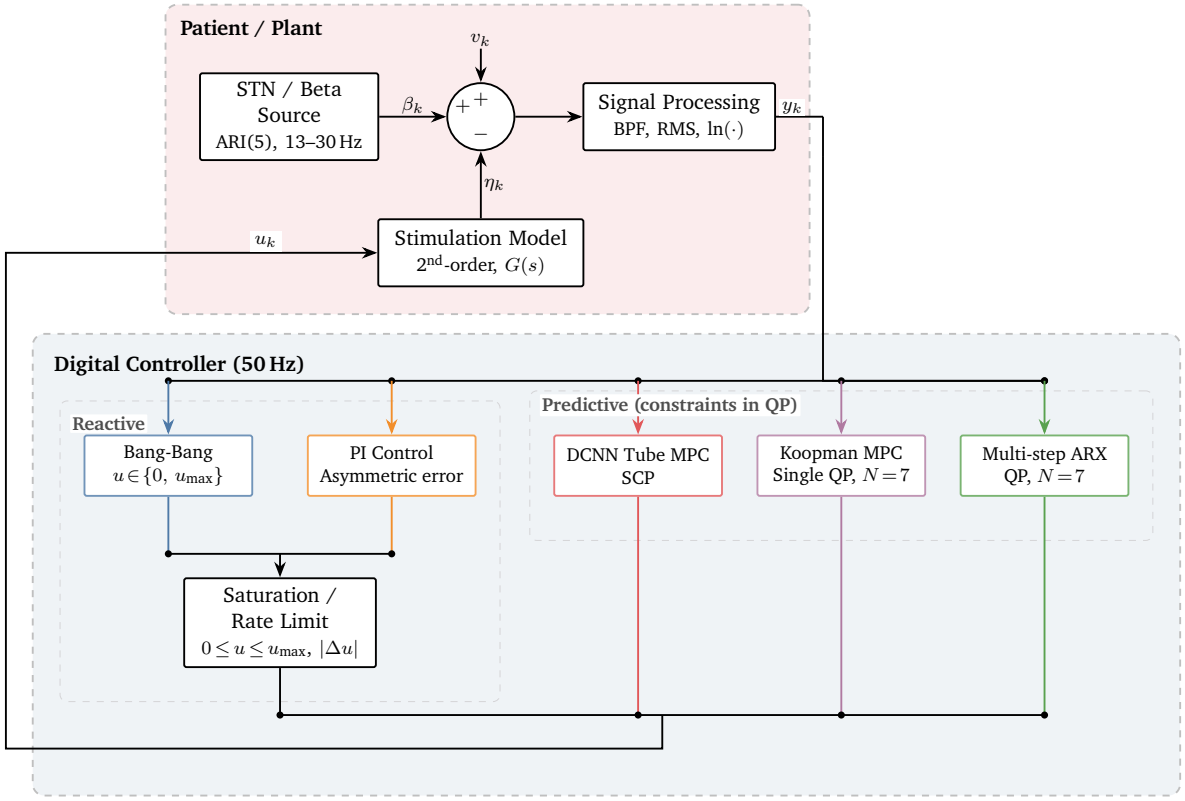

Figure 3.1: Closed-loop DBS system. Reactive controllers (bang-bang, PI) clip after the control law; MPC controllers (ARX, DCNN, Koopman) enforce limits inside the QP.

3.1.1 Log Transform

A log transform [13] turns the product into a sum:

$$y(t) = \ln(z(t)) \approx \underbrace{\ln(z_\beta(t))}_{\beta(t)} - \eta(t) + v(t)$$

where $v(t)$ is the noise term. In log space, the control goal is to keep $y(t)$ below $\beta_0 = \ln(z_0)$, where z_0 is the patient-specific beta-power threshold above which symptoms emerge.

This matters for learning: in raw space the DBS effect is <1% of total output, so MSE loss yields too little gradient to learn its tie to z_β. A linear-space DCNN captures only 11.5% of the true DBS effect at peak beta versus 95.9% in log space, cutting state-linked error from $R^2 = 16\%$ to 1.7%. All models use log space.

3.2 Stimulation Response Model

Following [12, 13], $\eta(t)$ is a second-order linear system:

$$\dot{x}_\eta(t) = A_\eta x_\eta(t) + B_\eta u(t), \qquad \eta(t) = C_\eta x_\eta(t)$$

with nominal values [5] $\bar{k} = 62.11$, $\bar{\tau}_1 = 0.05$ s, $\bar{\tau}_2 = 0.25$ s, and system matrices

$$A_\eta = \begin{bmatrix} -1/\tau_1 & 0 \\ 1/\tau_2 & -1/\tau_2 \end{bmatrix}, \quad B_\eta = \begin{bmatrix} k/\tau_1 \\ 0 \end{bmatrix}, \quad C_\eta = \begin{bmatrix} 0 & 1 \end{bmatrix}$$

The transfer function $G_\eta(s) = k/[(1+\tau_1 s)(1+\tau_2 s)]$ has real poles at $-1/\tau_1$ and $-1/\tau_2$.

For digital use at $T_s = 0.02$ s (50 Hz), zero-order hold gives

$$x_{\eta,k+1} = A_{\eta,d} x_{\eta,k} + B_{\eta,d} u_k$$

where $A_{\eta,d} = e^{A_\eta T_s}$ and $B_{\eta,d} = \int_0^{T_s} e^{A_\eta \tau} B_\eta \, d\tau$. Both discrete poles lie inside the unit circle.

3.3 Linearity Analysis

The envelope pipeline and the additive DBS coupling in log space both force a strongly linear form. This section first asks how much linearity the pipeline alone creates, then asks where true nonlinearity remains. The results guide the model designs that follow.

3.3.1 Pipeline-Induced Linearity

We extract the beta envelope via a 500 ms RMS window on the bandpass-filtered (13–30 Hz) signal, downsampled to 50 Hz, following the general approach of rectification and smoothing used in clinical adaptive DBS [3]. The 96% sample overlap acts as a strong low-pass filter. Feeding the pipeline inputs that no linear model can predict—white noise and a chaotic logistic map—and fitting one-step AR(15) to the outputs gives $R^2 = 0.92$ and $R^2 = 0.67$, both of which should be near 0. The gap is predictability manufactured by the smoothing, and sets the bar any AR fit on real data must clear.

Against this baseline, we fit an ordinary least squares (OLS) model on patient data. For $i = 1, \ldots, 5$,

$$\hat{y}_{k+i} = W_i \begin{bmatrix} y_{k-14} \\ \vdots \\ y_k \\ u_{k-14} \\ \vdots \\ u_k \\ u_{k+1} \\ \vdots \\ u_{k+5} \end{bmatrix} + b_i$$

where $W_i \in \mathbb{R}^{1\times 35}$ and $b_i \in \mathbb{R}$ (15 past log-beta, 15 past DBS, 5 future DBS). On held-out data, $R^2 = 0.9975$ at $i = 1$ falling to $R^2 = 0.921$ at $i = 5$. The $i = 1$ value beats both baselines, confirming real linear structure beyond the filter. But because the filter alone manufactures most of the R^2, the portion attributable to genuine dynamics is small, and whether any of it is nonlinear is the subject of the next subsection.

3.3.2 Residual Nonlinearity in Free Dynamics

The linear model fits most of the variance, but a Ramsey RESET test [18] rejects linearity at $i \geq 2$, pointing to a small but real nonlinear component in the free dynamics. Section 3.7 returns to the question of which features capture it.

The models that follow use this split. Koopman targets the free-dynamics leftover via lifted features; the DCNN learns a broad nonlinear map that can also catch state-control coupling.

3.4 Shared Setup for Data-Driven Models

The three data-driven predictors that follow (ARX, DCNN, Koopman) share a common state vector and are all fit on the same synthetically-augmented dataset.

3.4.1 State Vector

Each predictor acts on the state

$$z_k = [y_{k-n_y+1}, \dots, y_{k-1}, y_k,\ u_{k-n_u}, \dots, u_{k-1}]^T \tag{3.1}$$

where u_k is the DBS level at step k, holding n_y past outputs and n_u past inputs (oldest to newest). At 50 Hz with $n_y = n_u = 15$, $n_{\text{state}} = 30$, spanning 300 ms.

3.4.2 Training Data

Clinical data lack DBS, so training data are synthesised per [5]: a PRBS sets $\Delta u_k \in \{-\Delta u_{\max}, +\Delta u_{\max}\}$ and is applied to each patient's beta trace via the model of Section 3.2. After logging, state vectors z_k (3.1) and targets $y_{k+1}, \dots, y_{k+N}$ are formed by a sliding window.

Data use has three stages. First, pre-training on three training patients, then refinement on the test patient's training split, with a 10,000-sample test block held out behind 3,000-sample guard bands. Finally, controller evaluation runs on the held-out block only, so fitting and testing never overlap. Patient choice is in Section 6.1.1.

3.5 Multi-step ARX Baseline

The near-linearity above motivates a linear model. Steffen and Cannon [5, 13] found multi-step models beat single-step ones for DBS control; we adapt their approach to an ARX model on raw data with no state filter.

On the shared state z_k (3.1), the Multi-step ARX model fits one linear regressor per horizon step, with $N = 7$:

$$\hat{y}_{k+i} = W_i^{\text{ARX}} z_k + V_i^{\text{ARX}} u_{k:k+i-1} + b_i, \qquad i = 1, \dots, 7$$

where $W_i^{\text{ARX}} \in \mathbb{R}^{1\times 30}$, $V_i^{\text{ARX}} \in \mathbb{R}^{1\times i}$, and $b_i \in \mathbb{R}$ are trained by OLS on the data of Section 3.4.2. Since the model is affine in u, MPC reduces to a single QP per step (Section 4.3).

3.6 Multi-step DCNN Predictor

The DCNN replaces the linear ARX map with a learned nonlinear model on z_k (3.1), keeping the multi-step form of Section 3.5.

3.6.1 DCNN Architecture

Per [5], each horizon step uses a DC (difference-of-convex) split

$$f_i(z_k, u_{k:k+i-1}) = f_{i,1}(z_k, u_{k:k+i-1}) - f_{i,2}(z_k, u_{k:k+i-1})$$

where $f_{i,1}$, $f_{i,2}$ are input-convex neural networks (ICNNs) (Figure 3.2). Each ICNN has a free input layer (W_0, ReLU), one hidden layer with non-negative weights ($W_x \in \mathbb{R}^{n_h \times n_h}_{\geq 0}$), free skip links, and a non-negative output layer; non-negative weights with ReLU ensure convexity, and the DC form can fit any smooth map. A sweep over $n_{\text{hidden}} \in \{16, 32, 64, 128\}$ found 32 hidden units match larger nets at $\sim$18$\times$ less compute; we use $n_{\text{hidden}} = 32$.

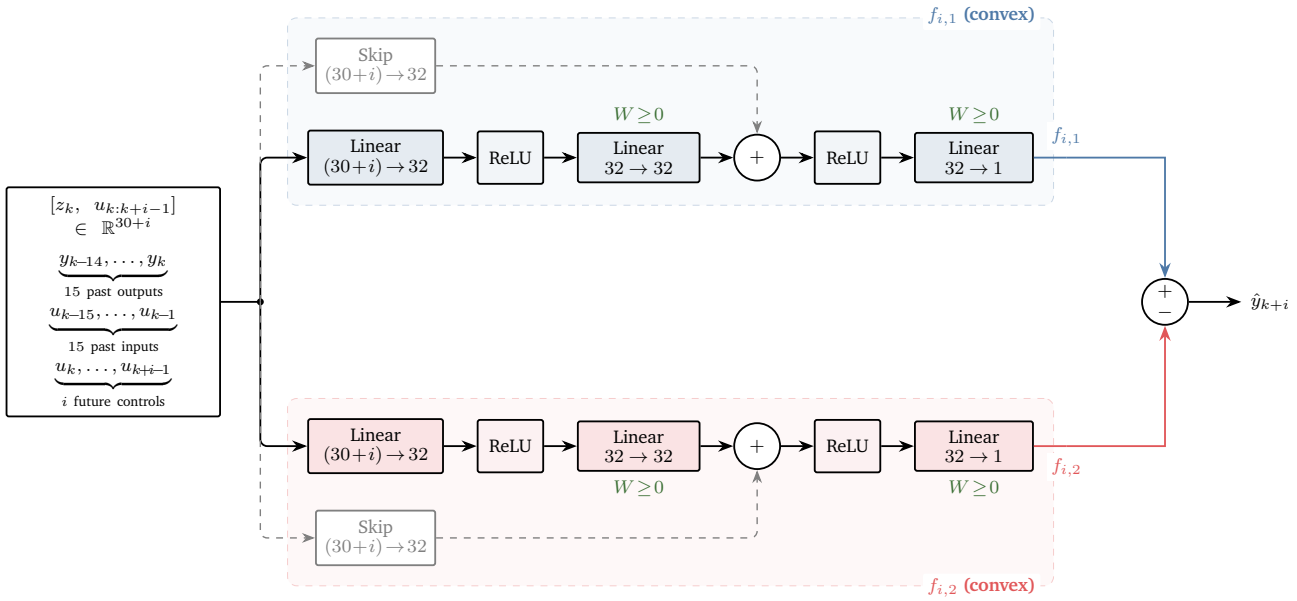

Figure 3.2: DCNN layout for one horizon step. Two ICNNs give $\hat{y}_{k+i} = f_1 - f_2$; one pair is trained per step $i = 1, \ldots, 5$.

3.6.2 Residual-DCNN Variant

A *Residual-DCNN* splits the prediction into a linear ARX base plus a learned DCNN correction: $\hat{y}_{k+i} = \hat{y}^{\text{ARX}}_{k+i} + \hat{r}^{\text{DCNN}}_{k+i}$, so the DCNN term handles only the nonlinear residual.

3.6.3 Network Training

Using the synthetic data of Section 3.4.2, each of the $N = 5$ nets is trained in PyTorch with MSE loss, Adam (lr 10^{-3}, linear warmup over 5 epochs then cosine decay), batch size 512, early stopping (patience 10) on an 80/20 split, and non-negative weight clips after each step.

3.6.4 Predictor Validation

Training curves for the test patient (Figure 3.3) show short horizons converge in 15 epochs and longer ones in 40–80. Residual state-linked error is $R^2 = 1.7\%$, confirming the log-space choice of Section 3.1.1 removes most bias.

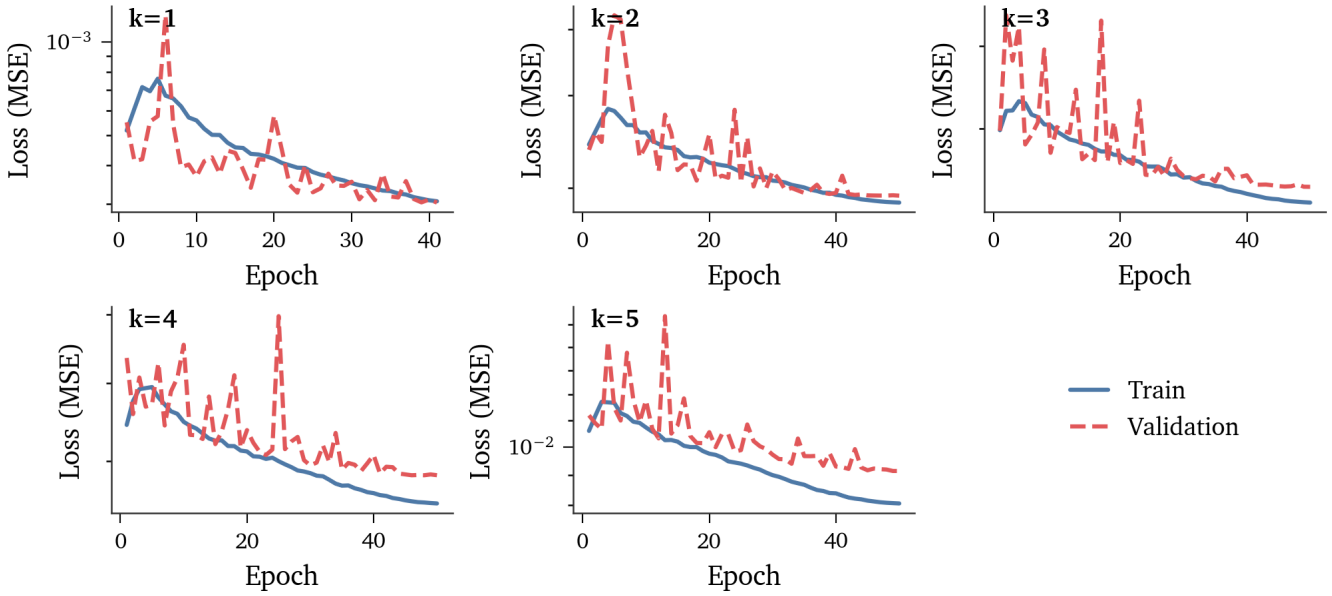

Figure 3.3: Training and test loss per horizon for the test patient ($n_{\text{hidden}} = 32$, log-space).

3.7 Koopman Predictor

The ARX baseline shows a linear multi-step model fits most of the dynamics. Koopman adds nonlinear lifting features while keeping outputs affine in control, targeting the leftover nonlinearity of Section 3.3 without breaking convexity.

Per Korda and Mezić [6] (Section 2.4.4), nonlinear lifting maps the free dynamics while future control enters linearly. In the simulated plant, DBS enters the log-beta dynamics linearly ($y = \beta - \eta$), so linear-in-u holds by design.

Unlike the autoencoder of Liang et al. [17], our output map C is a fixed selector matrix. Since $\psi(z) = [z,\ \phi(z)]$ keeps the raw state as a prefix, C simply picks out the current output entry, keeping outputs affine in u and allowing MPC to penalise the true output directly. An autoencoder Koopman model replaces C with a nonlinear decoder, breaking the affine form and forcing the controller to use a proxy cost or solve a nonlinear programme.

The linear-in-u form is untested in vivo—the DCNN, which allows nonlinear state-input coupling, is safer if this form is in doubt.

3.7.1 Model Architecture

A lifting map $\psi(z) = [z,\ \phi(z)]$ sends $z \in \mathbb{R}^{30}$ to $\mathbb{R}^{76}$ where dynamics are nearly linear:

$$\hat{y}_{k+i} = CA_i\psi(z_k) + CB_i\, u_{k:k+i-1}$$

with separate pairs (A_i, B_i) learned per step $i = 1, \ldots, 7$ and a shared output map C. Standard Koopman MPC steps the operator recursively; here, direct multi-step output avoids error build-up from an imperfect fit. Since $\psi(z_k)$ does not depend on future control, outputs are *affine in* u:

$$\hat{y}_{k+i} = e_i + F_{i,:}\,u, \qquad e_i = CA_i\psi(z_k), \quad F_{i,j} = CB_ie_j \tag{3.2}$$

where $F \in \mathbb{R}^{N\times N}$ is a constant lower-triangular Jacobian fixed at model load. Since $\psi(z_k)$ depends only on past data, all constraints stay linear in u regardless of how nonlinear ψ is. The encoder ϕ uses 46 features chosen by LASSO on forecast error, fed through a linear layer ($46 \to 46$); OLS trains the model on the shared data of Section 3.4.2.

Chapter 4

Controller Design

This chapter presents six controllers. Bang-bang and PI serve as reactive baselines; multi-step ARX MPC as a linear baseline. DCNN tube MPC and Residual-DCNN tube MPC use learned predictors in the same constrained framework. Koopman MPC exploits the linear control channel of the Koopman predictor (Section 3.7) to replace SCP with a single QP.

4.1 Bang-Bang Controller

The bang-bang controller raises stimulation by $\Delta u_{\max}$ when beta exceeds threshold, and lowers it otherwise:

$$u_{\text{on-off},k} = \begin{cases} \min\left(u_{\text{on-off},k-1} + \Delta u_{\max},\ u_{\max}\right) & \text{if } y_k > \beta_0 \\ \max\left(u_{\text{on-off},k-1} - \Delta u_{\max},\ 0\right) & \text{otherwise} \end{cases}$$

where $\Delta u_{\max}$ is the largest per-step change in amplitude, $u_{\max}$ the amplitude limit, and β_0 the patient-specific beta-power threshold. Bang-bang is the most advanced strategy in commercially available adaptive DBS devices and serves as the baseline for all ratios in Chapter 7.

4.2 Proportional-Integral Control

The PI controller uses a difference form for bounded updates with anti-windup:

$$u_{\text{PI},k} = u_{\text{PI},k-1} + \Delta u_{\text{PI},k}, \qquad e_k = [y_k - \beta_0]_+$$

$$\Delta u_{\mathrm{PI},k} = K_P \Delta e_k + K_I T_s e_k, \qquad \Delta e_k = e_k - e_{k-1}$$

We clamp the step to $[-\Delta u_{\max}, \Delta u_{\max}]$ and the input to $[0, u_{\max}]$.

4.3 Multi-step ARX MPC

Multi-step ARX MPC uses the linear ARX predictor of Section 3.5. Each horizon prediction $\hat{y}_{k+i} = W_i^{\mathrm{ARX}} z_k + V_i^{\mathrm{ARX}} u_{k:k+i-1} + b_i$ is affine in u, so the controller solves a single QP per timestep:

$$\min_{\mathbf{u},\mathbf{s}} \sum_{i=1}^{N} Q s_i^2 + \sum_{i=0}^{N-1} R\, u_{k+i}^2 \tag{4.1}$$

subject to:

$$\begin{aligned}
\hat{y}_{k+i|k} &= W_i^{\mathrm{ARX}} z_k + V_i^{\mathrm{ARX}} u_{k:k+i-1} + b_i && i = 1, \dots, N \\
s_i &\geq \hat{y}_{k+i|k} + w_i^{\max} - \beta_0, \quad s_i \geq 0 && i = 1, \dots, N \\
0 &\leq u_{k+i} \leq u_{\max} && i = 0, \dots, N-1 \\
|\, u_{k+i} - u_{k+i-1} \,| &\leq \Delta u_{\max} && i = 0, \dots, N-1
\end{aligned}$$

The prediction-error bound $w_i^{\max}$ is the same margin defined later in Section 4.4.1. As in the Koopman case, the predictor is affine in u. So the tube reduces to a one-sided shift of the threshold from β_0 to $\beta_0 - w_i^{\max}$, with no lower bound $w_i^{\min}$ or DC slacks. This controller shares the same 30-dimensional history state as all others and serves as the linear baseline.

4.4 DCNN Tube MPC

DCNN tube MPC replaces the linear predictor with the multi-step DCNN of Section 3.6, following the DC split and SCP framework of Steffen and Cannon [5].

4.4.1 DC Decomposition and Tube Bounds

Each i-step-ahead prediction splits into

$$\hat{y}_{k+i|k} = f_{i,1}(z_k, \mathbf{u}) - f_{i,2}(z_k, \mathbf{u})$$

where $f_{i,1}$ and $f_{i,2}$ are convex in $\mathbf{u} = u_{k:k+i-1}$ by the ICNN design (Section 3.6). At each SCP step the solver linearises the concave term but keeps the convex term exact. This confines the error to a single known term [15]. Steffen et al. [5] derive the full convex relaxation with tube variables $s_i^{\max}$ and $s_i^{\min}$, which bracket the true nonlinear prediction after linearisation.

A second layer of tightening comes from per-step prediction-error bounds $w_i^{\max} \geq 0$ and $w_i^{\min} \leq 0$. These bounds quantify how much the i-step-ahead prediction may differ from the plant. They enter the upper and lower DC constraints directly, so that $s_i^{\max}$ and $s_i^{\min}$ bracket the worst-case prediction rather than the nominal one. How the w_i are chosen—offline calibration or online adaptation—is deferred to Section 6.2. Figure 4.1 shows the resulting tube.

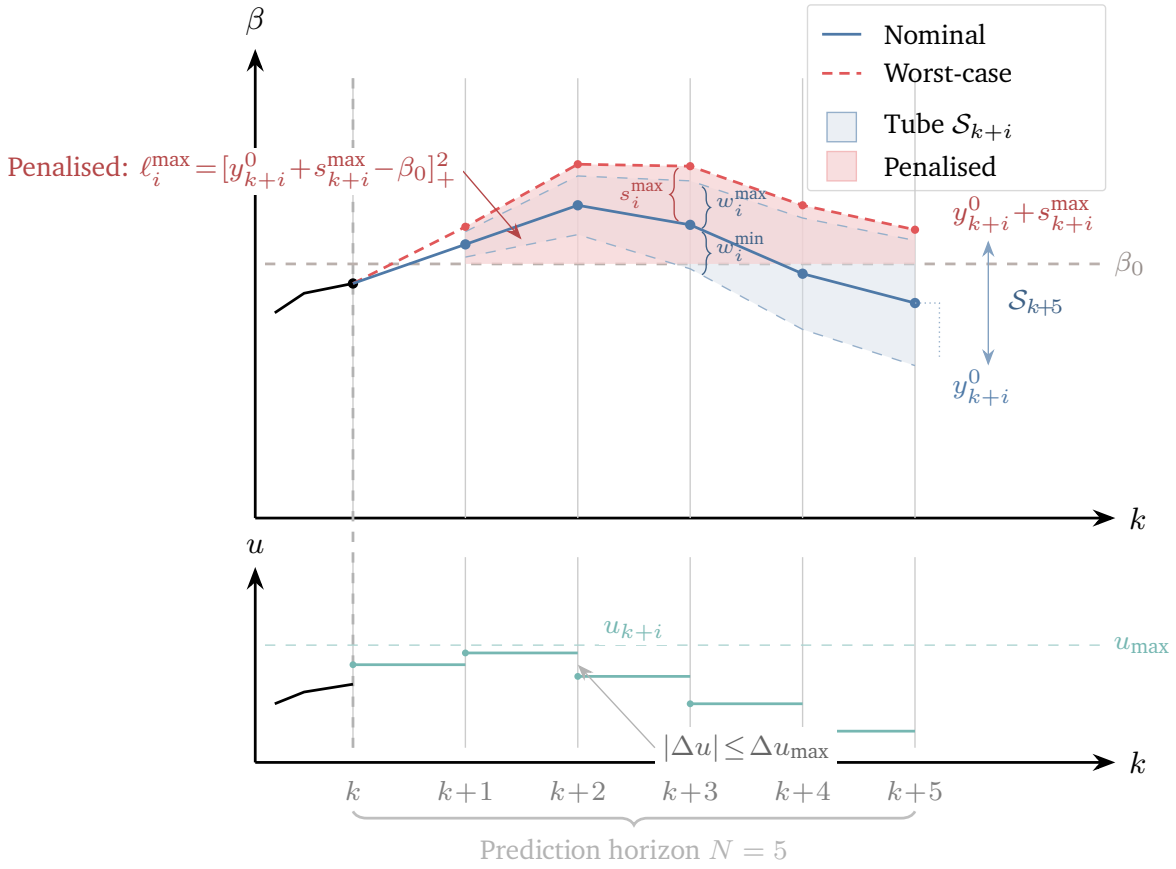

Figure 4.1: Tube MPC prediction over horizon $N = 5$. Top: nominal prediction with tube and worst-case upper bound driving the cost. Bottom: control sequence.

4.4.2 QP Subproblem

At each SCP iteration, the convex QP is:

$$\min_{\mathbf{u},\,\mathbf{s}^{\max},\,\mathbf{s}^{\min}} \sum_{i=1}^{N} Q\left[(y^0_{k+i|k} + s_i^{\max}) - \beta_0\right]_+^2 + \sum_{i=0}^{N-1} R\,u_{k+i}^2 \tag{4.2}$$

subject to the DC tube constraints

$$s_i^{\max} \geq f_{i,1}(\mathbf{u}) - \hat{f}_{i,2}(\mathbf{u};\mathbf{u}^0) - y^0_{k+i|k} + w_i^{\max} \tag{4.3}$$

$$s_i^{\min} \leq \hat{f}_{i,1}(\mathbf{u};\mathbf{u}^0) - f_{i,2}(\mathbf{u}) - y^0_{k+i|k} + w_i^{\min} \tag{4.4}$$

Here $\hat{f}(\mathbf{u};\mathbf{u}^0) := f(\mathbf{u}^0) + \nabla f(\mathbf{u}^0)^\top(\mathbf{u}-\mathbf{u}^0)$ is the linearisation of the concave side of the DC split around the SCP nominal $\mathbf{u}^0$. The term $y^0_{k+i|k} = f_{i,1}(\mathbf{u}^0) - f_{i,2}(\mathbf{u}^0)$ is the nominal prediction. Input bounds $0 \leq u_{k+i} \leq u_{\max}$, rate limits $|u_{k+i} - u_{k+i-1}| \leq \Delta u_{\max}$, and slack signs $s_i^{\max} \geq 0$, $s_i^{\min} \leq 0$ complete the problem. Only the worst-case upper path above β_0 enters the cost.

4.4.3 SCP Algorithm

The DC tube bounds depend on $\mathbf{u}$, making the full problem non-convex. SCP (Algorithm 1) solves QP (4.2) iteratively, linearising the concave side around the current solution until convergence [5, 16]. Warm-starting from the shifted prior solution keeps the iteration count low. If a QP is infeasible, the solver holds u_{k-1}.

Algorithm 1 DCNN tube MPC via SCP (per timestep k)

Require: State z_k, previous input u_{k-1}, warm-start $\mathbf{u}^0$, predictor $\{f_{i,1}, f_{i,2}\}$, bounds $\{w_i^{\max}, w_i^{\min}\}$, tolerances $\Delta J_{\min}$ and `maxiters`

$j \leftarrow 1$, $\Delta J \leftarrow \infty$, $J_0 \leftarrow \infty$
$y^0_{k+i} \leftarrow f_{i,1}(\mathbf{u}^0) - f_{i,2}(\mathbf{u}^0)$ for $i = 1, \ldots, N$
while $j \leq$ `maxiters` **and** $\Delta J > \Delta J_{\min}$ **do**
    Evaluate Jacobians $\nabla f_{i,1}(\mathbf{u}^0)$ and $\nabla f_{i,2}(\mathbf{u}^0)$
    Solve QP (4.2) for $\mathbf{u}_j^*$; if infeasible, **return** u_{k-1}
    $J_j \leftarrow J(z_k, \mathbf{u}_j^*)$, $\Delta J \leftarrow |J_j - J_{j-1}|$
    $\mathbf{u}^0 \leftarrow \mathbf{u}_j^*$, $y^0_{k+i} \leftarrow f_{i,1}(\mathbf{u}^0) - f_{i,2}(\mathbf{u}^0)$
    $j \leftarrow j + 1$
end while
return $\mathbf{u}^* = \mathbf{u}^0$

4.4.4 Residual-DCNN Tube MPC

Residual-DCNN-MPC uses the same framework but with the ARX + residual DCNN split of Section 3.6.2: $\hat{y}_{k+i|k} = \hat{y}^{\text{ARX}}_{k+i|k} + \hat{r}^{\text{DCNN}}_{k+i|k}$.

4.5 Koopman MPC

The DCNN and Residual-DCNN controllers place future inputs inside a nonlinear model, which requires iterative SCP. The Koopman predictor of Section 3.7 separates nonlinear state encoding from the linear control channel. This gives a single QP per timestep.

Prediction (3.2) is affine in u and the output map C is a fixed selection matrix. The lifted states can thus be eliminated to give $\hat{y} = e + Fu$, where e is the free response and F is a constant lower-triangular gain matrix. The controller solves (4.1) with $\hat{y}_{k+i|k} = e_i + F_{i,:}\,u$, subject to the same slack, input, and rate constraints as the multi-step ARX QP. The prediction-error bound $w_i^{\max}$ is the same margin defined for the DCNN controller (Section 4.4.1). The Koopman predictor is affine in u, so there is no SCP linearisation error. The tube collapses to a one-sided shift of the threshold from β_0 to $\beta_0 - w_i^{\max}$—no lower bound $w_i^{\min}$ and no DC slack variables are needed. Since F is fixed, the QP is built once per timestep with no iteration.

Table 4.1 summarises all six controllers.

Table 4.1: Summary of implemented controllers.

Property	**Bang-bang**	**PI**	**Multi-step ARX**	**DCNN-MPC**	**Res.-DCNN-MPC**	**Koopman MPC**
Continuous modulation	No	Yes	Yes	Yes	Yes	Yes
Predictive	No	No	Yes	Yes	Yes	Yes
Constraint handling	Implicit	Saturation	QP + ACI	Tube + ACI	Tube + ACI	QP + ACI
Model required	None	None	Linear ARX	DCNN	ARX + res. DCNN	Koopman encoder
Online adaptation	None	None	ACI bounds	ACI bounds	ACI bounds	ACI bounds
Prediction horizon‡	—	—	7 (140 ms)	5 (100 ms)	5 (100 ms)	7 (140 ms)
Decision variables	—	—	14	670	670	14

‡N=5 for DCNN (per [5]); N=7 for ARX/Koopman (no SCP cost).

Chapter 5

Implementation

Current DBS pulse generators are implanted in the chest and powered by a primary-cell battery lasting approximately 4 years [2]. Any closed-loop controller must therefore run within a tight power and compute envelope. The Medtronic Percept PC, the only adaptive DBS device now sold (Section 2.1), uses simple threshold switching; whether predictive MPC can fit within similar hardware constraints is an open question. Luo et al. [19] demonstrated closed-loop neural control on a Cortex-M4 at 120 MHz, showing the class is viable for real-time implantable processing. This chapter tests feasibility for MPC specifically, taking the DCNN-MPC and Koopman MPC controllers of Chapter 4 from a desktop prototype to an STM32L476RG Cortex-M4F (80 MHz, 128 KB SRAM, float32 FPU).

5.1 From Prototyping to Real-Time

The starting point is a CVXPY [20] prototype with a MOSEK backend. CVXPY puts the problem in standard form, checks Disciplined Convex Programming (DCP) rules, and rebuilds the solver input at every call. This is handy for testing but costly: one QP solve takes 72 ms median, and the full SCP loop averages 414 ms—exceeding the 20 ms budget by an order of magnitude, even before considering the far more limited chip target.

Calling MOSEK directly on persistent QP matrices (bypassing CVXPY's per-call canonicalisation) cuts single-QP time to 10.4 ms—a 6.8× gain. The full-size QP has two distinct numerical pathologies. First, the cost Hessian P is heavily rank-deficient: 640 of 670 variables are ReLU epigraph terms that carry no cost, so P has hundreds of zero eigenvalues. Second, the reduced KKT matrix is ill-conditioned ($\kappa \sim$

10^{11}): each ReLU constraint row contains a composition of weight matrices through the network, and the singular value spread of those rows compounds with depth. Together these rule out first-order methods (OSQP fails outright) and add overhead in MOSEK's general-purpose interior-point method. PIQP [21], a proximal interior-point solver, addresses both: its primal regulariser ρI restores positive-definiteness, and its bounded barrier weights keep κ finite (Section 5.3.1), giving 9 ms per QP. Yet even PIQP cannot meet the budget: the SCP loop averages 5.1 iterations with the 670-variable QP (Table 5.1), giving a total step time well over 40 ms on the desktop alone.

The bottleneck is therefore problem size, not conditioning per se. Stable-neuron elimination (Section 5.2) shrinks the QP to ∼30 variables (median 27–29 at 50 Hz, ∼40 worst case) by dropping the >98% of ReLU neurons whose activation status is provably fixed over the feasible input set. The surviving unstable neurons inherit the layer composition through their substituted constraint coefficients, so the per-row singular value spread—and hence κ—is largely preserved. The payoff is matrix dimension: a 30×30 KKT system makes PIQP's proximal regularisation cheap enough to meet the real-time budget. Section 5.3.4 contrasts this with the Koopman QP, which avoids the conditioning issue entirely.

5.2 Stable-Neuron Elimination

A ReLU neuron is *stable* if its activation status—active or inactive—cannot change over the feasible inputs; this is the neural-network usage of the term [22], unrelated to dynamical-systems stability. The tight rate bound ($|\Delta u_k| \leq 0.0024$ at 50 Hz) keeps interval-arithmetic bounds on each pre-activation narrow, so for most of the 640 DCNN neurons those bounds do not straddle zero and activation status is fixed across the entire feasible input set. Only the few *unstable* neurons, whose status can switch, need epigraph variables in the QP. Eliminating the stable neurons cuts the problem from 670 to ∼30 variables (median 27–29; max ∼40).

5.2.1 Neuron Classification via Interval Arithmetic

Each neuron's pre-activation a_j splits into a fixed part (depending on state) and a part linear in the control sequence. Splitting weight matrices into positive and negative parts and propagating the rate constraint $|\Delta u_k| \leq \Delta u_{\max}$ through each layer, interval arithmetic yields tight bounds $a_j^{\min}$ and $a_j^{\max}$ over all feasible inputs. The ICNN's non-negative weight constraint $W_x^{(\ell)} \geq 0$ gives monotone growth through layers, preventing interval blow-up. A safety margin $\varepsilon = 10^{-4}$ guards against numerical error at the classification

boundary:

$$\text{status}_j = \begin{cases} \text{stably-active} & \text{if } a_j^{\min} > \varepsilon \quad \Rightarrow \text{ReLU}(a_j) = a_j \text{ for all feasible } u \\ \text{stably-inactive} & \text{if } a_j^{\max} < -\varepsilon \quad \Rightarrow \text{ReLU}(a_j) = 0 \text{ for all feasible } u \\ \text{unstable} & \text{otherwise} \quad \Rightarrow \text{requires epigraph variable} \end{cases}$$

More than 98% of the 640 neurons are stably active or inactive at 50 Hz (median 7 unstable, max ∼17; rising at lower rates where $\Delta u_{\max}$ is larger). Figure 5.1 shows the scheme and Figure 5.2 the full pipeline.

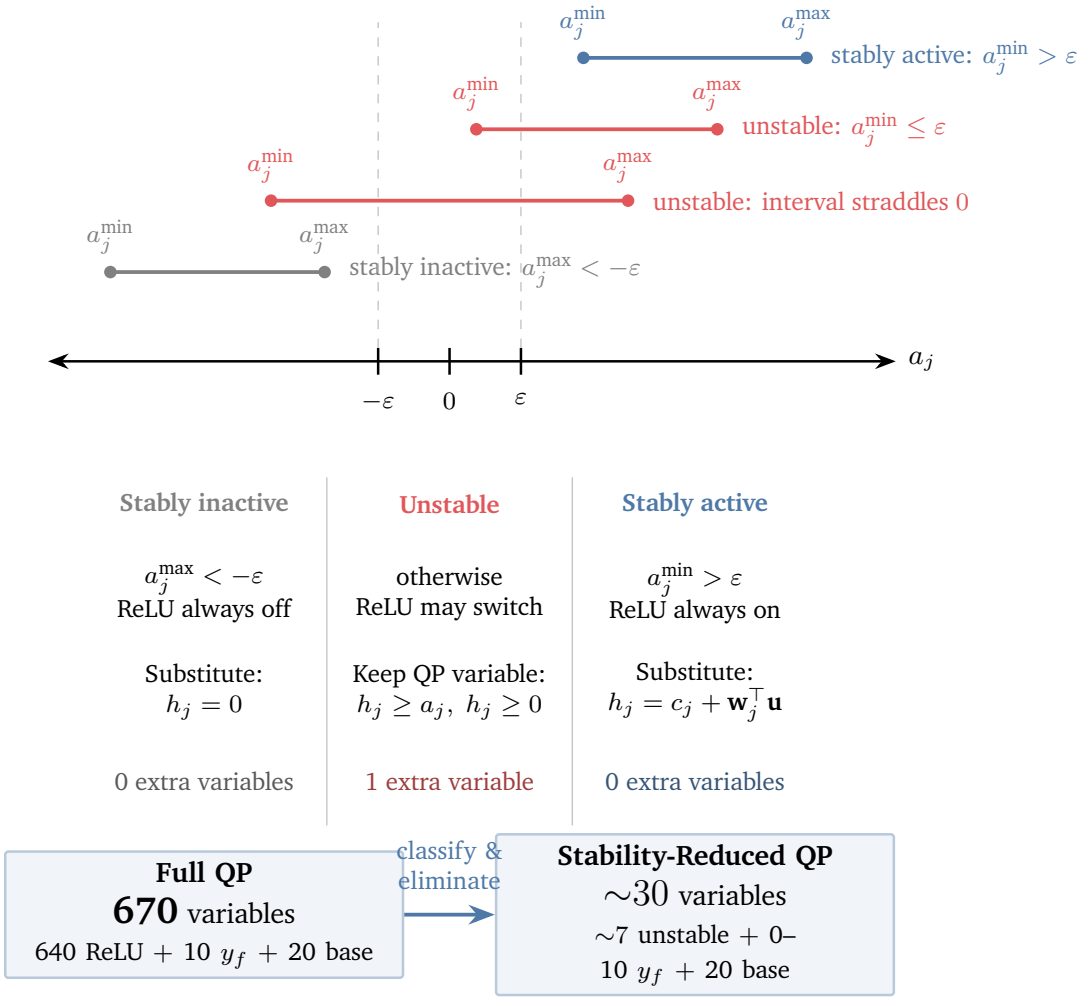

Figure 5.1: Neuron status by pre-activation bounds. Only the ∼1% unstable neurons (median 7 of 640 at 50 Hz) retain QP variables, cutting the problem from 670 to ∼30 variables.

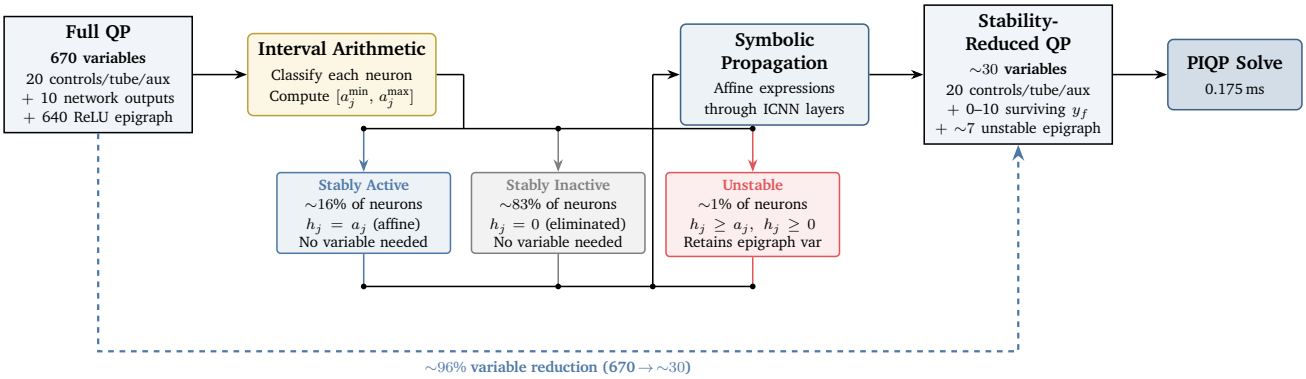

Figure 5.2: Stable-neuron elimination pipeline. Interval arithmetic and symbolic passes reduce the 670-variable QP to ~30 variables (median 27–29 at 50 Hz), solved by PIQP in 0.175 ms.

5.2.2 Symbolic Expression Propagation

Each neuron's output is then an affine map of the reduced vector $x \in \mathbb{R}^{n_{\text{vars}}}$:

$$h_j = h_j^{\text{const}} + (h_j^{\text{coeff}})^\top x$$

Stably active neurons keep the pre-activation; stably inactive ones give zero; unstable ones introduce an epigraph variable. Propagating layer by layer gives an affine output:

$$y_f = b_{\text{out}} + W_{\text{out}} h^{\text{const}} + (W_{\text{out}} H^{\text{coeff}})^\top x$$

which feeds into the QP cost and constraint matrices. When the last hidden layer for a given (step, network) has no unstable neurons, y_f is purely affine in the controls and is substituted directly into the tube cost—no y_f variable is kept. Across the QP this drops the always-present 6N=30 "core" to 4N=20 (controls, tube bounds, tracking epigraph), with up to 2N=10 surviving y_f variables added back on a per-step basis as needed.

We tested against the full CVXPY solver over 500 cases; the largest gap was 1.6×10^{-6}.

5.3 Embedded C++ Implementation

The desktop solver (PIQP/Eigen) assumes heap allocation, double precision, and unbounded code size—none of which the STM32L476RG provides. This section describes a C++ interior-point solver written from scratch for the target's 80 MHz Cortex-M4F, 128 KB SRAM, and float32-only FPU.

5.3.1 Interior-Point Solver

The neuron-reduced QP is solved by an interior-point method (IPM). The remainder of this section gives the reasons the desktop reference solver PIQP [21] cannot be used directly on the target, an overview of IPMs and the Mehrotra + PMM algorithm PIQP implements, and the float32 rewrite that replaces it.

Why a Bespoke Solver Is Needed

PIQP is written in C++ using Eigen. Three of its dependencies bar direct use on the STM32L476RG:

1. **Dynamic memory.** Eigen uses heap allocation (`malloc`); the firmware runs bare-metal with no heap and only 128 KB SRAM.
2. **Double precision.** PIQP assumes `double`; the Cortex-M4F has only a float32 FPU, so double arithmetic runs in software at $\sim 10\times$ cost.
3. **Code size.** Eigen's template instantiation alone exceeds the flash budget.

These are deep structural dependencies, not isolated calls that can be patched, so a full C rewrite is needed. The next two subsubsections set out the algorithm being ported and the changes float32 forces on it.

Interior-Point Method Overview

An IPM handles the inequalities $Gx \leq h$ by introducing slacks $s \geq 0$ with $Gx + s = h$, dual variables $z \geq 0$ for each row, and a log-barrier $-\mu \sum_i \log s_i$ that penalises iterates approaching $s = 0$. The barrier parameter μ is decreased across iterations; as $\mu \to 0$ the penalty vanishes and the iterate satisfies the original KKT conditions. At each μ the solver takes a Newton step on the perturbed KKT residual, giving the block system

$$\begin{bmatrix} P + \rho I & 0 & G^\top \\ 0 & Z & S \\ G & I & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta s \\ \Delta z \end{bmatrix} = - \begin{bmatrix} r_d \\ r_{sz} \\ r_p \end{bmatrix}, \tag{5.1}$$

where $S = \mathrm{diag}(s)$, $Z = \mathrm{diag}(z)$, and the centrality residual $r_{sz} = SZe - \sigma\mu e$ drives the iterates along the central path (e the all-ones vector, $\sigma \in [0, 1]$ the centering parameter). Progress is tracked by the duality gap $\mu = s^\top z / m$ over m inequalities. Given (5.1), the remaining design choices are how to pick σ each step and how to solve the block system efficiently.

Algorithm: Mehrotra Predictor-Corrector with PMM Regularisation

PIQP extends the baseline IPM with two refinements: Mehrotra's predictor-corrector heuristic [23] for choosing σ, and a Proximal Method of Multipliers framework [24] that regularises both the primal Hessian and the barrier weights.

Schur complement reduction. Eliminating Δs and Δz from (5.1) by Gaussian elimination of the second and third block rows yields an $n \times n$ system in Δx alone:

$$\underbrace{\left(P + \rho I + G^\top \Sigma G\right)}_{K} \Delta x = r, \qquad \Sigma = \operatorname{diag}(w_i),\ w_i = z_i / s_i. \tag{5.2}$$

This is the payoff of neuron reduction: $n \approx 30$ gives a 30×30 dense matrix factored by $\mathrm{LDL}^\top$ in $\mathcal{O}(n^3)$, trivially cheap next to the 670×670 full-size system. The eliminated updates Δs and Δz are recovered by back-substitution afterwards.

Predictor-corrector. A single Newton direction must guess σ in advance; Mehrotra chooses it adaptively using two solves per factorisation. The predictor solves (5.2) with $\sigma = 0$ to probe how far the iterates can move; from the resulting affine step sizes and predicted gap μ_{aff}, the rule $\sigma = (\mu_{\text{aff}}/\mu)^3$ yields small σ when the affine step made good progress (trust it) and near 1 otherwise (re-centre). The corrector reuses the same $\mathrm{LDL}^\top$ factor with a new right-hand side that adds the centering target $\sigma\mu$ and a second-order term $-\Delta s^{\text{aff}} \circ \Delta z^{\text{aff}}$ to cancel the SZe curvature. Two solves per factor roughly halve the iteration count versus a single-direction method.

PMM regularisation. The PMM framework contributes two regularisers, visible as ρ and a modified w_i in (5.2). The primal term ρI keeps K positive-definite even when P is singular—as it is here, because the epigraph block of P has many zero eigenvalues. The barrier weight $w_i = z_i/s_i$ in turn diverges as a constraint activates ($s_i \to 0$), so PMM replaces it with a bounded version:

$$w_i = \frac{z_i}{s_i + \delta\, z_i}, \qquad w_i \le \frac{1}{\delta}. \tag{5.3}$$

The barrier regulariser δ shrinks with μ (following PIQP, $\delta \mathrel{*}= (1 - \mu_{\text{rate}})$ per iteration), so the bounded-w perturbation of (5.2) vanishes at convergence. The primal regulariser ρ is held at a small fixed value (10^{-4} on the target, 10^{-8} on desktop) rather than decayed: it is already small relative to the problem scale and

would risk LDLT pivots falling below the float32 clamp (Section 5.3.1) if driven toward zero. The residual ρI bias is well below the IPM's stopping tolerance and does not affect the converged solution. The bounded w_i keeps the condition number of K finite throughout—critical for the float32 arithmetic below.

Embedded Reimplementation

The rewrite uses plain C-style loops over static arrays, with all sizes fixed at compile time. The core algorithm is unchanged—Mehrotra predictor-corrector with bounded-w weights—but float32 arithmetic and the limited SRAM force three categories of change.

Float32 numerical fixes. Running the textbook algorithm in float32 ($\varepsilon_{\text{mach}} \approx 1.2 \times 10^{-7}$) causes ∼40% of SCP steps to diverge. Two root causes were found:

1. **Barrier-weight overflow.** Even with the bounded w of (5.3), slacks s_i near the float32 floor create weights large enough to collapse the LDL$^\top$ diagonal $D_j = K_{jj} - \sum_{k<j} L_{jk}^2 D_k$. Clamping $w_i \geq 10^{-8}$ and the pivot $D_j \geq 10^{-6}$ prevents this.
2. **FMA rounding buildup.** The Cortex-M4F's fused multiply-add (`VMLA.F32`) alters rounding inside the same LDL$^\top$ update. In double precision the effect is negligible; in float32, ULP-level errors cascade through later columns and cause divergence. Compiling the solver with `-fno-fma` (keeping FMA elsewhere for speed) removes this.

Together these raise SCP convergence from ∼60% to 99.97%.

Dual recovery. PIQP recovers the dual update via $\Delta z_i = (r_i^s - z_i \Delta s_i)/s_i$ with $r_i^s = -s_i z_i - \Delta s_i^{\text{aff}} \Delta z_i^{\text{aff}} + \sigma\mu$; the division by s_i underflows in float32 as a constraint activates. The analytically equivalent form $\Delta z_i = w_i(-\Delta s_i + r_i^s/z_i)$ divides by z_i instead, which is safe in the opposite regime. We select the branch per constraint based on whichever of s_i, z_i is larger.

Sparsity exploitation. After condensing, G is ∼80% sparse: bound and rate rows have 1–2 nonzeros; only Jacobian rows are dense. We store G in both CSC and CSR layouts—CSC for the products $G^\top z$ and Gx, CSR for the row-wise rank-1 updates $K \mathrel{+}= \sum_i w_i\, g_i g_i^\top$ in the KKT build—cutting the $G^\top \Sigma G$ cost from $\mathcal{O}(n^2 m)$ to $\mathcal{O}(n^2 \bar{m})$, where $\bar{m}$ is the mean nonzeros per row. The LDL$^\top$ factor caches D_k^{-1} and $D_k L_{jk}$ per column, replacing costly `VDIV` calls and aiding cache reuse. Together these cut mean step time by 3.8× (160 ms → 42.6 ms on ARM).

5.3.2 Memory Architecture

Weights (∼130 KB) sit in flash; static arrays (∼67 KB) fill SRAM1; the IPM workspace uses a bump allocator in SRAM2 (31.9 KB). A ping-pong buffer cuts peak scratch use from 128 KB to 13 KB (Figure 5.3). All sizes are set at build time (`MAX_VARS`= 34, `MAX_AMB`= 10).

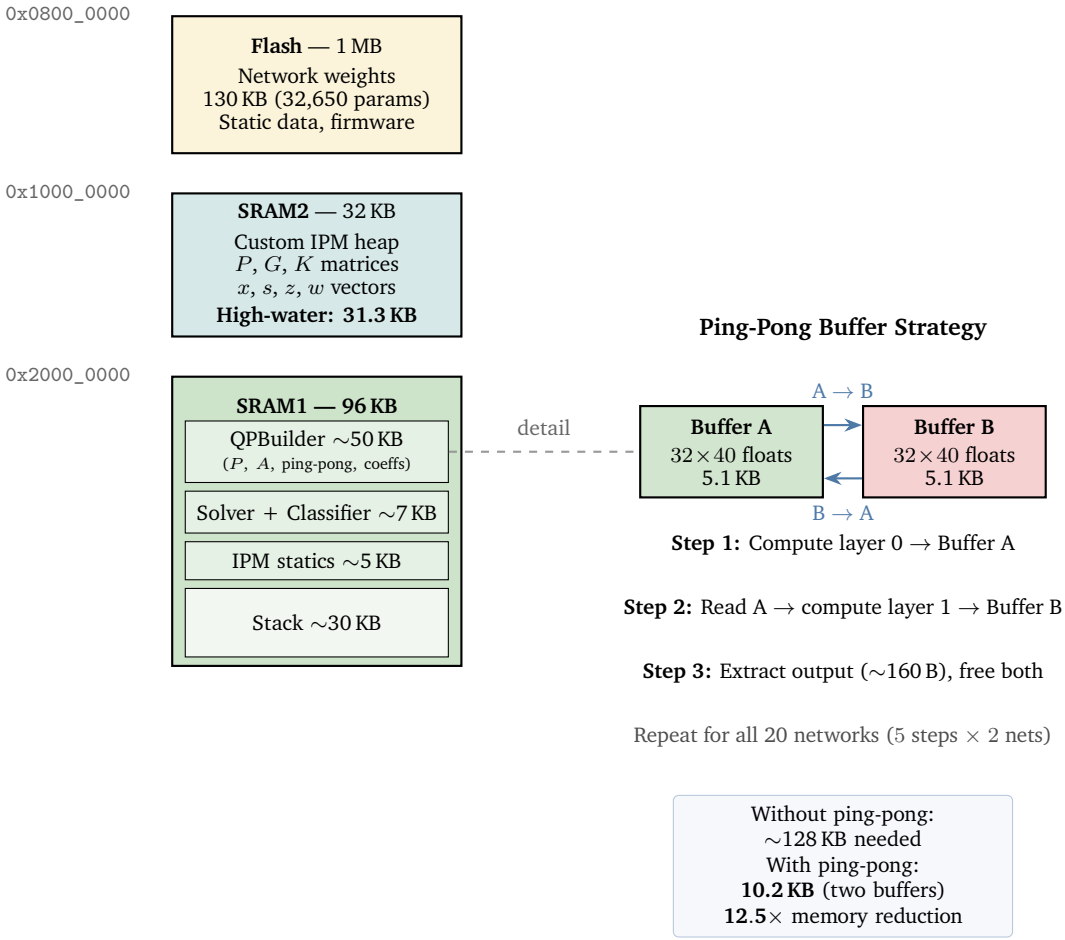

Figure 5.3: STM32L476RG memory layout. The ping-pong buffer scheme gives 12.5× peak memory savings versus naïve per-layer storage.

5.3.3 Hardware-in-the-Loop Validation

A Python host runs the patient model over UART; the firmware returns u^* with exact timing. Over 3,000 steps, the firmware achieves 99.97% SCP convergence and tracks within 2.1% of the double-precision Python reference on closed-loop suppression performance. Table 5.1 traces the solver from Python to the embedded target.

Holding the prior input for up to 3 sample periods still beats PI control. Compute power draw is 10.6 mW

Table 5.1: SCP solver performance across versions. Median step time includes neuron status check, QP build, solve, and convergence check.

Implementation	QP Vars	SCP Iters	Median Step
CVXPY full QP (MOSEK)	670	5.1	—
Python neuron-reduced (PIQP)	∼30	2.1	0.175 ms
C++ desktop (embedded IPM, f64)	∼30	1.3	—
STM32L476RG ARM (embedded IPM, f32)	∼30	1.3	41.0 ms

at 10 Hz and 1.4 mW at 3 Hz, both within known DBS budgets [2].

5.3.4 MPC Formulation Comparison

Both controllers run on the same embedded IPM (Section 5.3.1); what differs is the MPC formulation around it. The Koopman form (Section 4.5) gives a single QP per timestep, removing the SCP loop, and presents the solver with a smaller problem (14 variables, 42 constraints). The encoder requires only a 46×46 matrix-vector product, using 10.7 KB of flash (55× less than the DCNN's 590 KB). Ruiz scaling and primal padding ($\epsilon_P = 0.1$) ensure float32 convergence.

Table 5.2 compares both controllers on the STM32L476RG. Removing the SCP loop and shrinking the QP cuts IPM steps, giving 4.3× speedup. At 9.9 ms mean step time, the Koopman controller fits within the 20 ms budget for 50 Hz control. Power draw is 4.9 mW at 50 Hz, well within known DBS power budgets [2].

Unlike the DCNN subproblem, the Koopman QP is well-conditioned: its Hessian is $\mathrm{diag}(2R\,I,\ 2Q\,I)$ with both blocks carrying meaningful cost, so there are no near-zero eigenvalues from zero-cost epigraph terms. The PMM regularisation essential for the DCNN's ill-conditioned system (Section 5.3.1) is therefore not load-bearing here. Further solver optimisation is likely available—active-set or first-order methods such as OSQP, which fail on the DCNN problem, would be viable for the Koopman QP. We reuse the existing IPM because it already meets the timing budget and was validated on the target; exploring alternatives is left to future work.

Table 5.2: Embedded performance comparison on STM32L476RG (80 MHz, float32). Both controllers share the same embedded IPM (Section 5.3.1); the single-QP Koopman formulation removes SCP and cuts step time by 4.3×.

Metric	**DCNN-MPC (SCP)**	**Koopman MPC (single QP)**
QP variables	∼30	14
QP constraints	∼60	42
SCP iterations	1.3	—
IPM iterations / step	∼16.5	4.3
Mean step time	42.6 ms	9.9 ms
Median (P50)	41.0 ms	9.5 ms
P95	69.2 ms	12.8 ms
Max	∼70 ms	20.4 ms
P95 / P50 ratio	1.69	1.34
Feasibility	99.97%	100%
Flash (weights)	590 KB	10.7 KB
Firmware size	219 KB	57 KB

Chapter 6

Evaluation Methodology

This chapter describes how the controllers of Chapter 4 are tested: the simulation setup, online bound methods for tube MPC, tuning, and the 50-trial evaluation behind the results of Chapter 7.

6.1 Simulation Framework

6.1.1 Patient Data

From 30 DBS recordings (no stimulation), seven are selected with sufficient length (≥ 15 min) and signal quality. One patient serves as the primary test case for model development and controller tuning, three form a pre-training cohort with time-split data (Section 3.4.2), and three are reserved for cross-patient evaluation (Section 7.4). Key parameters for the test patient: 50 Hz sampling, $\beta_0 = 1.93$ (75th percentile), $u_{\max} = 0.030$, $\Delta u_{\max} = 0.0024$. Figure 6.1 shows the burst pattern behind the one-sided cost.

6.1.2 Closed-Loop Simulation

The loop (Algorithm 2) takes the measured beta as the base signal and adds the stimulation effect in log space (Section 3.2).

All predictive controllers build z_k from past data. The plant values (g, τ_1, τ_2) are fixed within each 15 s trial but drawn afresh from ±40% of nominal. Since trials use distinct beta segments, the draws are also distinct.

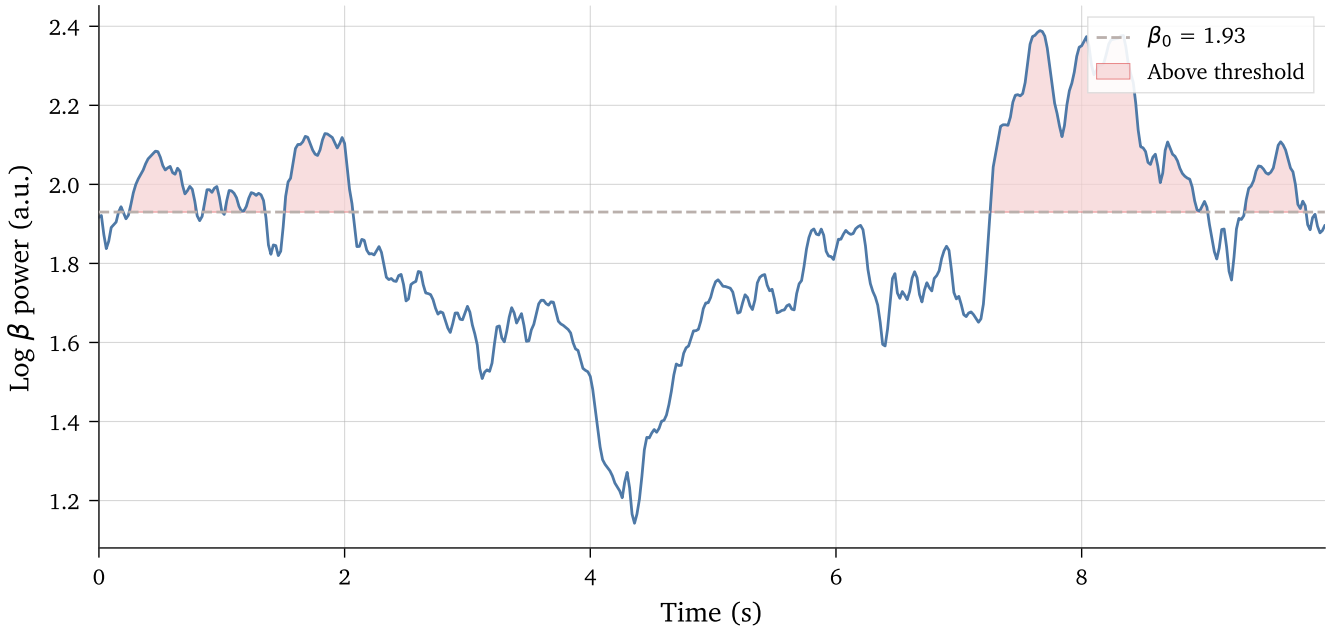

Figure 6.1: Ten seconds of log-beta from the test patient. The threshold $\beta_0 = 1.93$ (dashed) marks the onset of excess beta; shaded regions show bursts that the controller must suppress.

Algorithm 2 Closed-loop DBS simulation

Require: Patient data $\{y_k^{\text{nom}}\}$, controller, stimulation model parameters (g, τ_1, τ_2)

1: Initialise stimulation state $(\eta_0, \dot{\eta}_0) = (0, 0)$, control history $u_{-1} = 0$
2: **for** $k = 1, \ldots, N_{\text{steps}}$ **do**
3: Update stimulation effect: $\eta_k = \eta_{k-1} + T_s\dot{\eta}_{k-1}$, $\dot{\eta}_k = \dot{\eta}_{k-1} + T_s(-a_1\dot{\eta}_{k-1} - a_0\eta_{k-1} + g \cdot u_{k-1})$
4: Apply stimulation: $y_k = y_k^{\text{nom}} - \eta_k$
5: Construct state: $z_k = [y_k, \ldots, y_{k-n_y+1}, u_{k-1}, \ldots, u_{k-n_u}]$
6: Compute control: $u_k = \text{Controller}(z_k, u_{k-1})$ with saturation and rate constraints
7: **end for**
8: Compute metrics after discarding 5-second warmup

6.1.3 Performance Metrics

We discard the first 5 seconds (250 steps at 50 Hz) as warmup, allowing the AR state and control history to settle from zero and giving the online bound trackers (DKW, ACI, PAC) enough prediction-error samples to initialise. After warmup, three metrics are computed:

$$\text{Tracking error:} \quad \bar{e} = \frac{1}{N}\sum_{k=1}^{N}\left[\max(0, y_k - \beta_0)\right]^2$$

$$\text{Mean stimulation:} \quad \bar{u} = \frac{1}{N}\sum_{k=1}^{N} u_k$$

$$\text{Time above threshold:} \quad T_{\text{above}} = \frac{1}{N}\sum_{k=1}^{N} \mathbf{1}[y_k > \beta_0]$$

We state results as ratios to bang-bang: error ratio $= \bar{e}_{\text{ctrl}}/\bar{e}_{\text{BB}}$, input ratio $= \bar{u}_{\text{ctrl}}/\bar{u}_{\text{BB}}$.

6.2 Online Disturbance Bounds

Tube MPC requires per-step disturbance bounds $W_i = [w_i^{\min}, w_i^{\max}]$ quantifying how much the model's i-step-ahead prediction may differ from the plant. Wider bounds make the controller more cautious (higher stimulation to guarantee safety); tighter bounds save energy but risk violations if the true error exceeds the bound. Steffen and Cannon [5] computed these bounds offline as the 80th-percentile absolute prediction error over training data. Fixed bounds cannot adapt: when the model predicts well they waste stimulation energy; when predictions deteriorate—due to parameter drift or patient-state change—they may be too tight and permit violations. This section evaluates three methods for updating the bounds online as new prediction errors arrive.

Throughout, let $e_k^{(i)}$ denote the i-step prediction error at time k. After n steps the controller has a running sample $\{e_1^{(i)}, \ldots, e_n^{(i)}\}$ for each horizon step i; each method uses this sample to set w_i.

6.2.1 Dvoretzky–Kiefer–Wolfowitz (DKW) Bounds

The DKW inequality [25, 26] bounds the entire error distribution via a non-parametric confidence band. It states that the empirical CDF F_n from n samples converges uniformly to the true CDF F:

$$P\left(\sup_x |F_n(x) - F(x)| > \varepsilon\right) \leq 2\exp(-2n\varepsilon^2).$$

Inverting at confidence $1-\delta$ gives a band of width $\varepsilon = \sqrt{\ln(1/\delta)/(2n)}$. We implemented two modes. *Sample-max mode* sets $w_i = \max_{k \le n} |e_k^{(i)}|$, guaranteed to contain future errors from the same distribution (up to DKW confidence), but conservative: one outlier locks the bound high permanently. *Quantile mode* keeps only the most recent W errors and tracks the $(1-\alpha)$-quantile of this sliding window, so old outliers age out.

The key limitation is that the DKW guarantee assumes errors are i.i.d. draws from a stationary distribution [25], which may not hold under the distribution shifts that motivate online bounds.

6.2.2 Probably Approximately Correct (PAC) Concentration Bounds

Rather than bounding the full distribution, PAC concentration inequalities bound the mean error and add a confidence margin that shrinks as more samples arrive. Tube MPC only needs the bound to hold on average for stability (Lyapunov decrease in expectation), so targeting the mean can yield tighter bounds.

We tested three classical inequalities. Let $\bar{e}_i = \frac{1}{n}\sum_{k=1}^{n} |e_k^{(i)}|$ be the sample mean of absolute errors, $\hat{V}_i$ the sample variance, and $\hat{\sigma}_i$ the sample standard deviation. The constant C is an upper bound on the error magnitude ($|e_k^{(i)}| \le C$ for all k), set to the largest absolute error in the offline calibration data.

- **Bernstein** [27]: $w_i = \bar{e}_i + \sqrt{2\hat{V}_i \ln(3/\delta)/n} + 3C\ln(3/\delta)/n$. Uses variance for a tighter margin when errors are concentrated, with a secondary heavy-tail term.
- **Cantelli**: $w_i = \bar{e}_i + k\,\hat{\sigma}_i$, where $P(e > w_i) \le 1/(1+k^2)$; $k = 2$ gives at most 20 % exceedance.
- **Hoeffding**: $w_i = \bar{e}_i + C\sqrt{\ln(1/\delta)/(2n)}$. Loosest bound, but requires no variance estimate—only the range C.

All three assume independent, identically distributed errors from a fixed distribution and require a known bound C on the error magnitude. Like DKW, these assumptions break under distribution shift.

6.2.3 Adaptive Conformal Inference (ACI)

ACI [28] takes a different approach: instead of estimating the error distribution, it maintains a threshold per horizon step and adjusts it online based on violations. The threshold $\theta_k^{(i)}$ is updated as:

$$\theta_{k+1}^{(i)} = \max\Big(0,\ \theta_k^{(i)} + \gamma\left(\mathbf{1}[|e_k^{(i)}| > \theta_k^{(i)}] - \alpha\right)\Big),$$

where $\alpha \in (0, 1)$ is the target miss rate and $\gamma > 0$ controls adaptation speed. A miss ($\mathbf{1}[\cdot] = 1$) raises the threshold by $\gamma(1 - \alpha)$; a hit lowers it by $\gamma\alpha$. This drives the long-run miss rate asymptotically to α for *any* error sequence, with no i.i.d. requirement [28].

We set $\alpha = 0.2$ (80 % coverage) and $\gamma = 0.005$. Thresholds are initialised from the offline bounds and take over once sufficient errors are observed during warmup (Section 6.1).

6.2.4 Method Comparison

Table 6.1 summarises the three methods. ACI has two advantages. First, its coverage guarantee holds under distribution shift without i.i.d. errors, whereas DKW and PAC assume stationarity not strictly satisfied by autocorrelated residuals. Second, ACI requires less tuning: α and γ map directly to coverage rate and adaptation speed, whereas DKW and PAC need a window size, confidence level δ, and—for PAC—a choice of inequality and error range C. Section 7.2 confirms ACI gives the best error–input trade-off; it is used for all main results.

Table 6.1: Online disturbance bound methods compared.

Property	**DKW**	**PAC**	**ACI**
Guarantee type	CDF bound	Mean + margin	Coverage rate
Requires i.i.d. errors	Yes	Yes	No
Handles distribution shift	Via windowing	Via windowing	Inherent
Key parameters	δ, W, α	δ, C, k	α, γ

6.3 Controller Tuning

Each controller has two free parameters: K_p and K_i for PI, and tracking weight Q and input penalty R for MPC. To avoid evaluating on the full dataset at every combination, we tune via grid search on six 15-second windows spanning the range of beta dynamics in the training data.

Chapter 7

Results and Discussion

Following Steffen and Cannon [5], we evaluate each controller on 50 distinct 15 s trials (750 samples at 50 Hz) drawn from the held-out P1 test block. The 50 trials start at different offsets into the test block, so each controller faces a diverse range of beta dynamics rather than a single favourable window. Each trial also applies randomised parameter mismatch (± 40 % peak shift in g, τ_1, τ_2; see Section 6.1), testing robustness to patient variability. Seeds are shared across controllers for paired comparison; error bars show 95 % confidence intervals.

7.1 50-Trial Controller Comparison

7.1.1 Error-Stimulation Trade-off

The key question is whether nonlinear features improve the error-input trade-off over the Multi-step ARX baseline. Setting tuning aside, we swept the cost weights Q and R for each MPC method and traced the Pareto frontier: the best tracking error at each input level.

Figure 7.1 shows the result. For this patient, Koopman-MPC consistently outperforms DCNN-MPC, which in turn outperforms Multi-step ARX across most of the frontier: at a given input budget, the nonlinear methods reach lower tracking error. In the input range 0.75–0.85, Koopman-MPC gets 5–8% lower error than DCNN-MPC at matched input, and 8–15% lower than Multi-step ARX. All three share the same 30-dim history state and multi-step prediction. The frontier gap shows that nonlinear features help beyond what a linear multi-step model can reach.

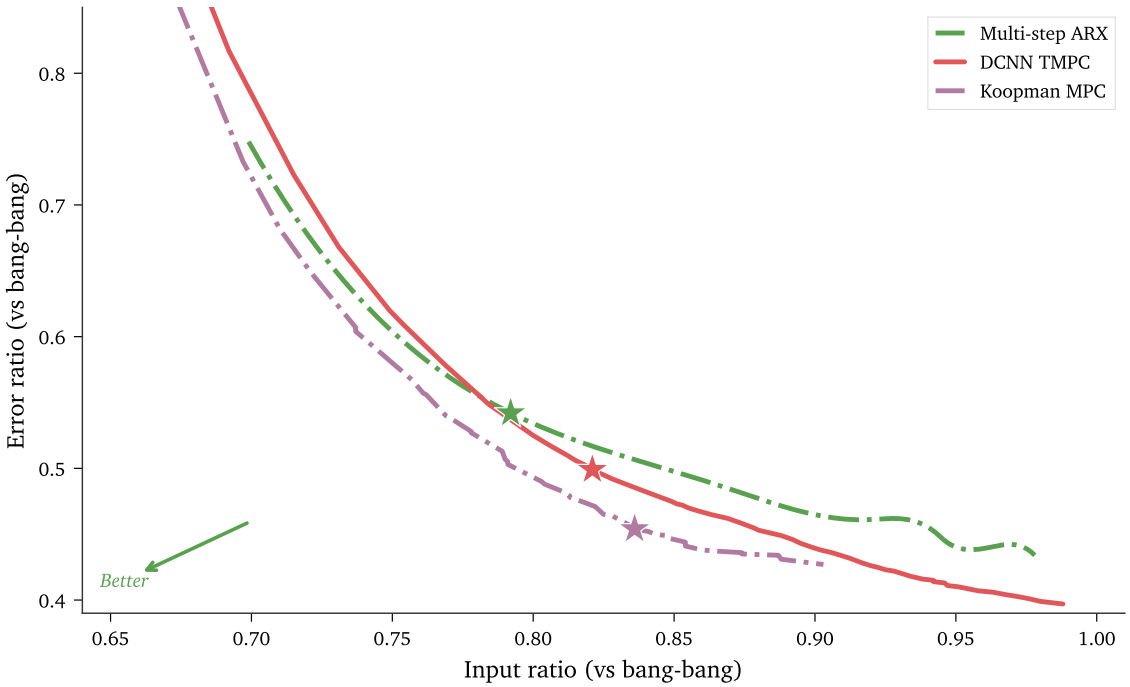

Figure 7.1: Pareto frontiers for three MPC methods on P1. Stars mark the cost-optimal operating point. Lower-left is better.

7.1.2 Cost-Optimal Operating Points

For all further tests, each method is tuned to minimise a shared quadratic cost $J = Q_{\text{eval}}\, e + R_{\text{eval}}\, \bar{u}^2$, where e is the mean-squared tracking error and $\bar{u}^2$ the mean squared stimulation input, with fixed evaluation weights chosen so that both terms contribute equally at the bang-bang baseline. The cost-optimal Q/R is then the MPC weight pair that minimises J/J_{BB}. Table 7.1 lists the resulting points; Table 7.2 gives the raw values.

Table 7.1: Ratios relative to bang-bang (50-trial means, best-cost Q/R)

Controller	Error Ratio	Input Ratio
Koopman-MPC	**0.437**	0.876
Multi-step ARX	0.468	0.890
Residual-DCNN-MPC	0.505	0.840
DCNN-MPC	0.507	**0.834**
PI	0.900	1.178

Table 7.2: Mean ± std. dev. across 50 trials (patient P1, ±40% parameter drift)

Metric	Bang-bang	PI	Multi-step ARX	DCNN-MPC	Koopman-MPC	Res.-DCNN-MPC
Tracking Error ($\times 10^{-4}$)	10.39 ± 13.08	9.35 ± 14.37	4.86 ± 7.43	5.27 ± 7.83	$\mathbf{4.54 \pm 7.16}$	5.24 ± 7.76
Mean Stimulation ($\times 10^{-4}$)	15.60 ± 18.48	18.37 ± 22.51	13.88 ± 15.43	$\mathbf{13.02 \pm 14.61}$	13.66 ± 15.10	13.10 ± 14.58

Koopman-MPC has the lowest normalised cost, with error ratio 0.437 and input ratio 0.876. Multi-step ARX tracks nearly as well (0.468) but uses more input (0.890). DCNN-MPC and Residual-DCNN-MPC reach similar cost at different operating points: DCNN-MPC uses the least input (0.834) with mid-range tracking (0.507). PI matches bang-bang tracking (0.900) but uses 18% more input, with no gain. Paired Wilcoxon signed-rank tests on the 50 matched segments confirm that the tracking-error differences between all controller families are statistically significant ($p < 10^{-4}$). All MPC variants beat bang-bang on every trial, despite never being tuned on drifted data.

7.1.3 Representative Traces

Figure 7.2 shows closed-loop traces for one beta segment, showing how each method responds to the same shifts.

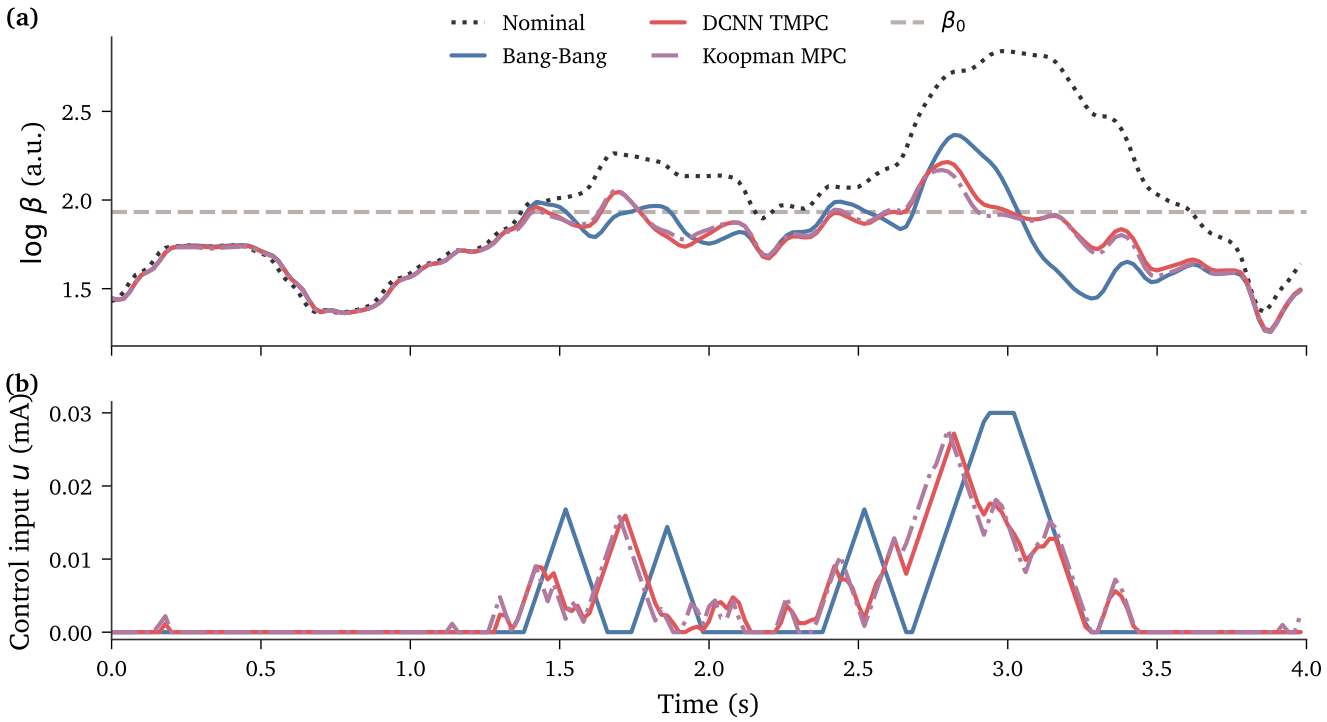

Figure 7.2: Closed-loop traces for bang-bang, Multi-step ARX, DCNN-MPC, and Koopman-MPC on the same beta segment. (a) Beta with threshold β_0 and nominal (no control) trace. (b) Control inputs. PI and Residual-DCNN-MPC data are in Tables 7.1 and 7.2.

7.2 Online Bound Behaviour

Of the four methods, DKW sample-max was the most conservative (input ratio > 2), driven by a single large residual. PAC Cantelli was less conservative, but its fixed shape could not keep both tracking error

and input usage low simultaneously. DKW windowed quantile gave the lowest input ratio (0.73) at the cost of its formal coverage guarantee. ACI matched this adaptivity (input ratio ≈ 0.80) while retaining marginal coverage. Its bounds settle at 40–60% of the offline bounds, and it was used for all subsequent results.

7.3 Drift Robustness Analysis

Can the controller maintain performance as the patient's physiology changes, or does it need online model updates? This section shows that it does not: the predictor's autoregressive structure absorbs drift passively, and five online adaptation methods fail to improve on the baseline.

7.3.1 Drift Scenarios

We test three drift scenarios, each applied at step 750 (15 s into the trial). Together they span signal-side changes (gain, medication cycle) and plant-side changes (stimulation efficacy). Table 7.3 summarises the cases.

Table 7.3: Drift scenarios applied at step 750.

Scenario	**Mechanism**	**Magnitude**	**Clinical basis**
Gain	Beta amplitude scaling	1.3×	Dopaminergic fluctuation [11]
Medication	Amplitude × frequency	1.4×, 0.85×	Levodopa wear-off [29]
Stim efficacy	Reduced plant gain g	62.1 → 43.5 (−30%)	Impedance growth [30]

7.3.2 Prediction Under Signal Drift

The processed beta envelope has a lag-1 autocorrelation of 0.997, a direct consequence of the 500 ms RMS window with 96% sample overlap. An AR(15) model fit to this signal has output weights summing to $\sum W_y = 0.996$—a near-unit root. A permanent level shift of δ produces a predicted shift of 0.996δ; the model tracks the new level rather than reverting to the training mean. This explains drift immunity for the Multi-step ARX and the linear component of the Residual-DCNN (whose ARX weights sum to 0.994). For the DCNN, no analogous weight decomposition exists—it is a nonlinear function of z_k—so the evidence is empirical.

Figure 7.3 provides that evidence: a +0.2 log-unit step is applied to a test segment at step 500. All four models show a single-step error spike that returns to baseline within 3–4 steps—the predictor re-anchors

to the shifted signal almost immediately, whether linear or nonlinear.

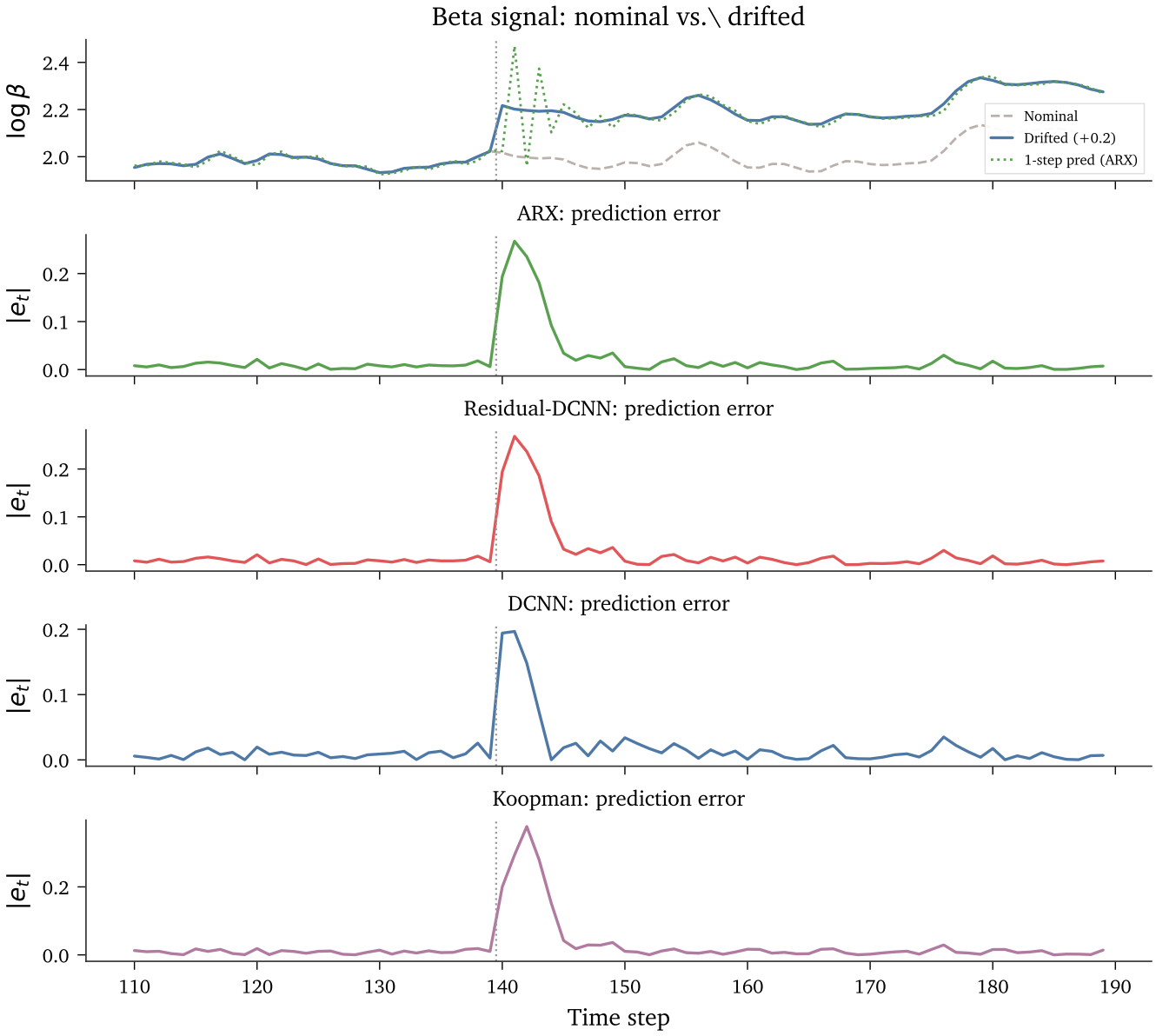

Figure 7.3: Step response test: +0.2 log-unit shift at step 500. Top: drifted signal with ARX prediction overlay. Lower panels: absolute prediction error per model.

Open-loop tests (no controller) across the three drift scenarios confirm the pattern. Gain drift leaves prediction error unchanged (PE ratio ≈ 1.00 at $k = 1$ for ARX, DCNN, Residual-DCNN). Medication drift *improves* prediction (PE ratio ≈ 0.77 at $k = 1$) because the $0.85\times$ frequency reduction smooths the signal—first-difference variance drops 30%, making it easier to predict. Stim efficacy drift does not alter the beta signal at all (it changes the plant gain g), so PE ratio $= 1.000$ exactly. The one exception is Koopman-MPC: its LASSO-selected lifting features include nonlinear transforms that are not shift-invariant, and gain drift raises PE by 4–12% across the horizon. This is the trade-off for its richer representation.

Stim efficacy drift is different in kind. It reduces the plant gain (g drops 30%), so each unit of stimulation produces less effect. In closed loop, the controller responds correctly—it increases input to compensate—but cannot fully offset the loss due to rate and amplitude caps, and the suppression rate drops 39.8%. No predictor update can fix a weaker actuator.

A 2×2 factorial test (drift on/off $\times$ ACI on/off) confirms that the adaptive tube bounds contribute at most +1.2% to drift handling; the AR structure is the source. Five online adaptation methods (drift-triggered AR-only, drift-triggered AR+FIR, always-on AR+FIR, drift-triggered dither+ARX, always-on dither+ARX) gave tracking-error gaps $< 0.5\%$ (Cohen's $d < 0.3$). Always-on ARX was actively harmful, warping the near-unity weight sum and driving input to 19.2$\times$ baseline.

7.3.3 Paired Parameter Mismatch

The drift scenarios above test specific clinical shifts. To check that controller rankings hold under general parameter uncertainty, we ran each of the 50 segments twice: once with nominal parameters and once with per-trial values drawn from $\pm 40\%$ of nominal. All four MPC methods and bang-bang were tested.

Figure 7.4 plots the result. Error and input ratios (each normed by bang-bang on the same trial) sit tightly on the $y{=}x$ line: parameter mismatch does not shift any controller's operating point. Drift magnitude has no notable correlation with the error-ratio change ($|r| < 0.2$, $p > 0.18$ for all controllers).

7.4 Cross-Patient Evaluation

The results in Section 7.1 evaluate on a single patient. To check whether the pipeline works across patients, we ran the full training-tuning-testing steps on three additional patients. None were in the neural network pre-training set. For each patient, Koopman-MPC and Multi-step ARX were retrained by OLS on that patient's data, while DCNN-MPC was fine-tuned from the same pre-trained model (trained on P5–P7). Cost weights Q/R were swept per patient and method. Each patient was evaluated over 50 trials using held-out test data with $\pm 40\%$ randomised parameter mismatch. Signal quality is measured by the *variance ratio*: total beta envelope variance over residual variance after a 0.5 s moving average. A higher value indicates a less noisy envelope with more power in the slow dynamics that the controller can act on.

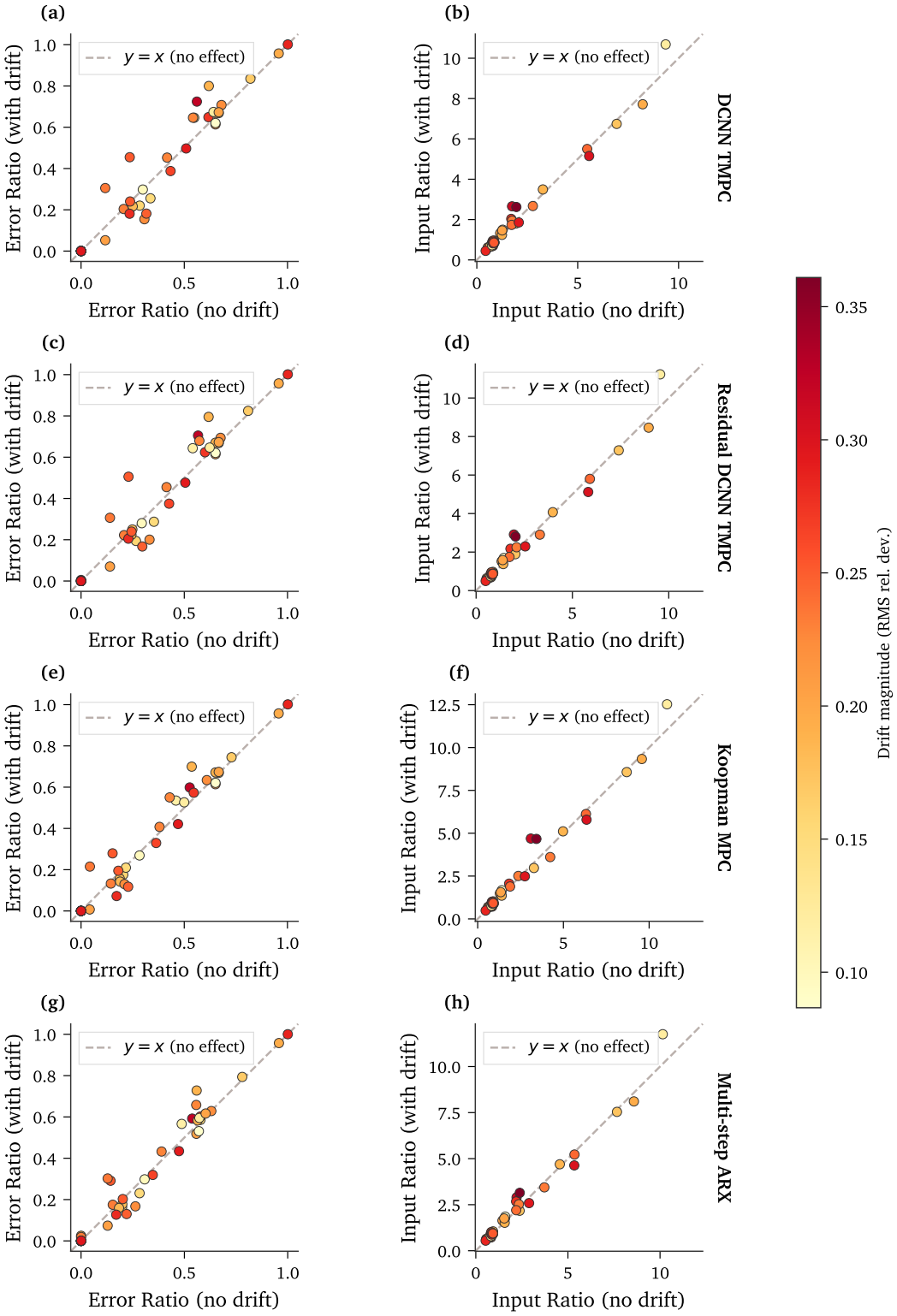

Figure 7.4: Paired parameter mismatch test (nominal vs. ±40% drift) for four MPC methods over 50 trials. Colour encodes drift magnitude.

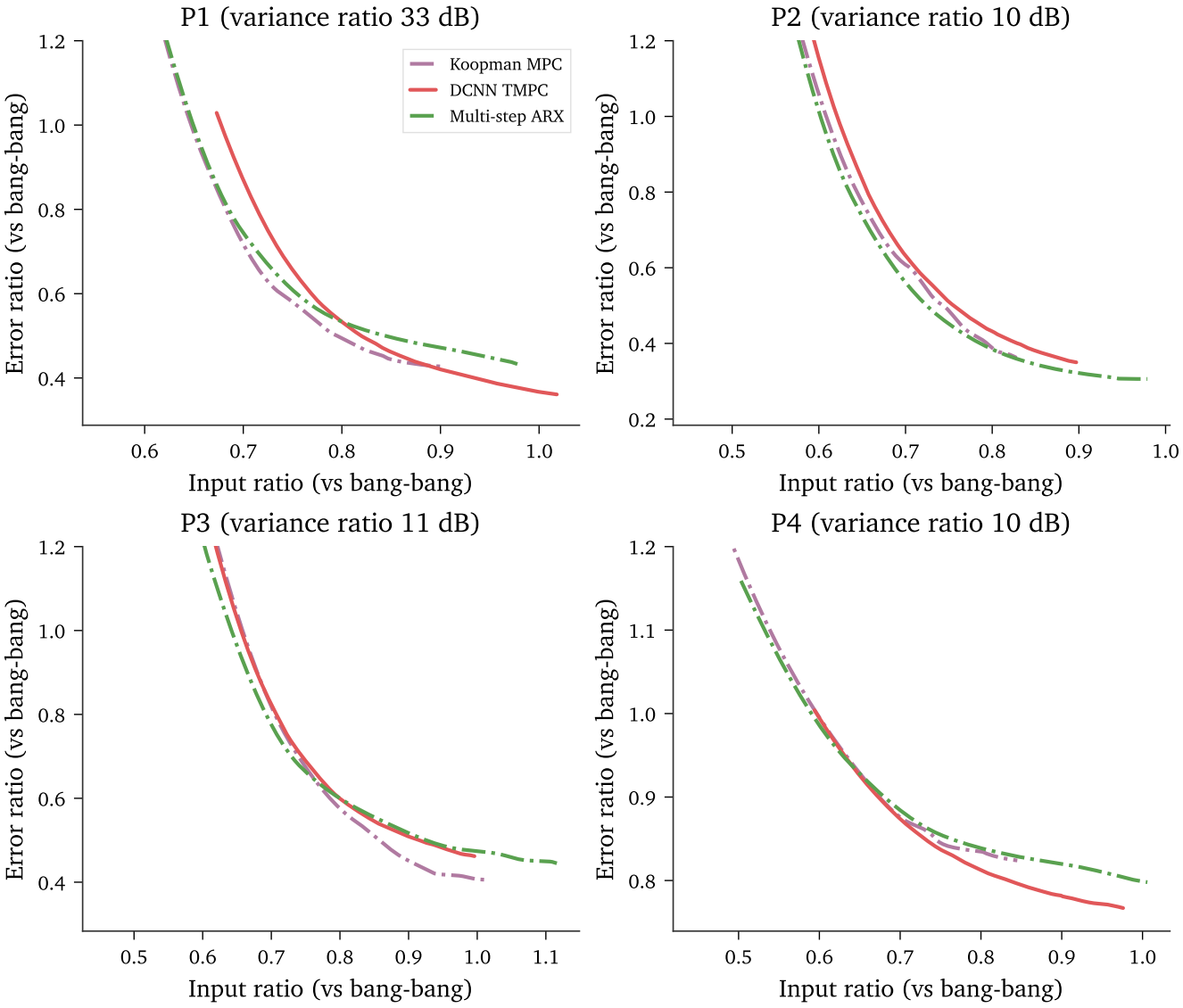

Figure 7.5: Pareto frontiers for three MPC methods across four patients. Lower-left is better.

7.4.1 Pareto Frontiers Across Patients

Figure 7.5 shows the Pareto frontiers. Every MPC method beats bang-bang on every patient, confirming that the pipeline generalises. Absolute performance varies—P1 (variance ratio 33 dB) reaches error ratios below 0.45, while P4 (variance ratio 10 dB) plateaus near 0.80—but this cross-patient spread is expected given the wide range of signal characteristics reported in the STN-LFP literature [9, 11].

The method ranking, however, is patient-dependent (Table 7.4). On P1, Koopman-MPC and DCNN-MPC improve on Multi-step ARX. On P2, ARX and Koopman-MPC are closely matched and both outperform DCNN-MPC. On P3 and P4 the methods are tightly bunched at low input ratios, but diverge at higher input: Koopman-MPC pulls ahead on P3 while DCNN-MPC does so on P4.

7.4.2 Why Rankings Differ

Two structural factors explain the variation.

Koopman-MPC and Multi-step ARX share the same QP setup ($N = 7$, exact solve, ACI bounds) and differ only in the lifted feature set (76 features with 46 LASSO-chosen nonlinear terms vs. 31 with intercept). When the nonlinear features add nothing, the two methods converge—as on P2, where their frontiers nearly overlap. On P1 and P3, the nonlinear terms improve prediction (MAE reduction of 3–5% at $k = 7$) and the frontier shifts accordingly. The benefit of nonlinear lifting scales with the nonlinearity present in each patient's dynamics.

DCNN-MPC benefits from pre-training on other patients, which acts as implicit regularisation. On low-variance-ratio patients (P2–P4) where patient-specific data is noisier, this regularisation effect is strongest. On P4 the benefit is large enough for DCNN-MPC to pull ahead at higher input ratios; on P2, it is not, and ARX and Koopman-MPC match or beat DCNN-MPC.

Table 7.4: Cross-patient summary. Variance ratio measures signal quality; Pareto outcome summarises Figure 7.5.

Patient	Var. ratio (dB)	Pareto outcome
P1	33.1	Koopman/DCNN lead
P2	10.4	ARX/Koopman matched
P3	11.2	Koopman leads at high input
P4	9.6	DCNN leads at high input

Chapter 8

Conclusions

8.1 Summary

This project brought nonlinear tube MPC for closed-loop DBS from desktop simulation to implant-grade hardware. Across 50 trials with ±40% randomised parameter mismatch, Koopman-MPC achieved the lowest tracking error (0.437 versus bang-bang) and a 9.9 ms embedded step time—well within the 50 Hz budget. PI control offered no improvement over bang-bang, confirming that a predictive horizon matters here.

Three observations stand out.

- The log-beta dynamics are near-linear ($R^2 = 0.997$ at $k = 1$), yet the small nonlinear residual still matters: it cuts the error–input trade-off by 4–12%, amplified in closed-loop by tighter ACI bounds. Better open-loop prediction does not always help—up to 12.8% lower prediction error can leave the Pareto frontier unchanged, as robustness margins absorb moderate model differences.
- Drift robustness is a non-problem for signal-level changes: the AR structure of the smoothed beta envelope provides passive robustness, and online adaptation adds less than 0.5%. Plant-side drift is different—a 30% loss in stimulation efficacy costs 39.8% suppression, and no predictor update can fix actuator saturation.
- Log-space training proved essential. Without it, the stimulation effect is invisible to the loss function and prediction bias correlates with state.

The cross-patient evaluation (four patients, variance ratio 10–33 dB) confirmed that every MPC design

beats bang-bang on every patient, but the ranking is patient-specific. Where dynamics are more linear, lifting features add nothing and Koopman reduces to ARX. The value of nonlinear models depends on the patient's signal, not the method alone.

On the implementation side, there is no accuracy–speed trade-off. Stable-neuron elimination reduced the DCNN QP by ∼96%, and the Koopman formulation then replaced the SCP loop with a single QP per step, cutting step time from 42.6 ms to 9.9 ms at 4.9 mW and 55× less flash. Koopman-MPC is both the best controller and the fastest solver.

8.2 Contributions

The main methodological contribution is **stable-neuron elimination for ICNN-based MPC** (Section 5.2): an exact scheme using interval arithmetic to classify each ReLU neuron as permanently active or inactive and remove it from the QP. It applies to any optimisation over an input-convex neural network with bounded inputs.

The remainder of the project applies and adapts existing techniques to build a complete closed-loop DBS system:

1. **Embedded deployment.** A custom float32 IPM on STM32L476RG hardware, with numerical fixes for float32 precision (Section 5.3).
2. **Koopman MPC for DBS.** The Koopman framework of Korda and Mezić [6] adapted with LASSO-selected lifting features and a linear output map, replacing the SCP loop with a single QP (Sections 3.7, 4.5 and 5.3.4).
3. **Disturbance bounds and drift analysis.** Comparative evaluation of DKW, PAC, and ACI bound methods; factorial analysis of drift robustness sources (Sections 6.2 and 7.3).
4. **Paired 50-trial evaluation.** Comparing six controllers under matched randomised parameter mismatch (Section 7.1).

8.3 Limitations

Three limitations constrain what this work can claim.

- **Simulation scope.** All results share a single plant model fit to population-averaged stimulation response data (Section 3.2). The true response varies with electrode site, impedance, and disease course. Relative rankings remain valid, but absolute scores may not transfer in vivo. Cross-patient tests cover four patients on one recording system (TMSi Saga32, 4096 Hz).
- **Koopman linearity assumption.** Koopman MPC assumes input enters the dynamics linearly. This holds by construction in the log-space simulation but has not been tested in vivo; if it fails, the DCNN's nonlinear-in-u capacity provides a fallback.
- **Validation gaps.** Hardware-in-the-loop testing confirms numerical match and timing margins but excludes real-time LFP filtering, artefact rejection, telemetry, and clinical safety interlocks.

8.4 Future Work

Each limitation points to a natural extension.

- **Larger patient cohorts.** A larger, multi-device cohort would test whether the signal-quality-linked method ranking generalises and whether per-patient LASSO feature selection improves on the fixed dictionary used here.
- **Tighter stable-neuron classification.** The interval-arithmetic bounds are conservative. Tighter bound propagation from the neural network verification literature would classify more neurons as stable for the same input set, further shrinking the QP.
- **Signal processing and clinical integration.** Real-time bandpass filtering, envelope extraction, and artefact rejection must fit within the timing budget before clinical use. In-vivo validation, fail-safe monitoring, and regulatory approval then follow; the modular architecture allows each stage to be tested independently.

The core question was whether tube MPC with a learned predictor could run on implant-grade hardware while outperforming reactive baselines. Across six controllers, four patients, and 200 randomised trials, the answer is yes. The remaining distance to clinical use lies not in the control algorithm but in surrounding infrastructure: signal processing, safety interlocks, and in-vivo validation.

References

[1] E. R. Dorsey, T. Sherer, M. S. Okun, and B. R. Bloem, "The emerging evidence of the parkinson pandemic," *J. Parkinson's Dis.*, vol. 8, no. s1, pp. S3–S8, 2018.

[2] D. J. Lee, C. S. Lozano, R. F. Dallapiazza, and A. M. Lozano, "Current and future directions of deep brain stimulation for neurological and psychiatric disorders," *J. Neurosurg.*, vol. 131, no. 2, pp. 333–342, 2019.

[3] S. Little, A. Pogosyan, S. Neal, B. Zavala, L. Zrinzo, M. Hariz, T. Foltynie, P. Limousin, K. Ashkan, J. FitzGerald, A. L. Green, T. Z. Aziz, and P. Brown, "Adaptive deep brain stimulation in advanced parkinson disease," *Ann. Neurol.*, vol. 74, no. 3, pp. 449–457, 2013.

[4] S. Stanslaski, R. L. S. Summers, L. Tonder, Y. Tan, M. Case, R. S. Raike, N. Morelli, T. M. Herrington, M. Beudel, J. L. Ostrem, S. Little, L. Almeida, A. Ramirez-Zamora, A. Fasano, T. Hassell, K. T. Mitchell, E. Moro, M. Gostkowski, N. Sarangmat, and H. Bronte-Stewart, "Sensing data and methodology from the ADAPT-PD clinical trial," *npj Parkinson's Dis.*, vol. 10, no. 1, p. 174, 2024.

[5] S. Steffen and M. Cannon, "Deep learning model predictive control for deep brain stimulation in parkinson's disease," in *Proc. IEEE Conf. Decision Control (CDC)*, 2025, pp. 5744–5749.

[6] M. Korda and I. Mezić, "Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control," *Automatica*, vol. 93, pp. 149–160, 2018.

[7] L. V. Kalia and A. E. Lang, "Parkinson's disease," *Lancet*, vol. 386, no. 9996, pp. 896–912, 2015.

[8] A. L. Benabid, S. Chabardes, J. Mitrofanis, and P. Pollak, "Deep brain stimulation of the subthalamic nucleus for the treatment of parkinson's disease," *Lancet Neurol.*, vol. 8, no. 1, pp. 67–81, 2009.

[9] G. Tinkhauser, A. Pogosyan, S. Little, M. Beudel, D. M. Herz, H. Tan, and P. Brown, "The modulatory

effect of adaptive deep brain stimulation on beta bursts in parkinson's disease," *Brain*, vol. 140, no. 4, pp. 1053–1067, 2017.

[10] Medtronic plc, "FDA approval for adaptive deep brain stimulation (Percept PC)," https://news.medtronic.com/2025-02-24-Medtronic-earns-U-S-FDA-approval-for-the-worlds-first-Adaptive-deep-brain-stimulation-system-for-people-with-Parkinsons, 2025, accessed 24 March 2025.

[11] A. A. Kühn, A. Kupsch, G.-H. Schneider, and P. Brown, "Reduction in subthalamic 8–35 Hz oscillatory activity correlates with clinical improvement in Parkinson's disease," *Eur. J. Neurosci.*, vol. 23, no. 7, pp. 1956–1960, 2006.

[12] J. E. Fleming, E. Dunn, and M. M. Lowery, "Simulation of closed-loop deep brain stimulation control schemes for suppression of pathological beta oscillations in parkinson's disease," *Front. Neurosci.*, vol. 14, p. 166, 2020.

[13] S. Steffen, M. Cannon, H. Tan, and J. Debarros, "Model predictive control for closed-loop deep brain stimulation," in *Proc. IEEE Conf. Decision Control (CDC)*, 2024, pp. 4034–4039.

[14] B. Amos, L. Xu, and J. Z. Kolter, "Input convex neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 146–155.

[15] M. Doff-Sotta and M. Cannon, "Difference of convex functions in robust tube nonlinear MPC," in *Proc. IEEE Conf. Decision Control (CDC)*, 2022, pp. 3044–3050.

[16] Y. Lishkova and M. Cannon, "A successive convexification approach for robust receding horizon control," *IEEE Trans. Autom. Control*, vol. 70, no. 10, pp. 6436–6448, 2025.

[17] Z. Liang, Z. Luo, K. Liu, J. Qiu, and Q. Liu, "Online learning Koopman operator for closed-loop electrical neurostimulation in epilepsy," *IEEE J. Biomed. Health Inform.*, vol. 27, no. 1, pp. 492–503, 2023.

[18] J. B. Ramsey, "Tests for specification errors in classical linear least-squares regression analysis," *J. R. Stat. Soc. B*, vol. 31, no. 2, pp. 350–371, 1969.

[19] J. Luo, D. Firfilionis, M. Turnbull, W. Xu, D. Walsh, E. Escobedo-Cousin, A. Soltan, R. Ramezani, Y. Liu, R. Bailey, A. O'Neill, A. S. Idil, N. Donaldson, T. G. Constandinou, A. Jackson, and P. Degenaar, "The Neural Engine: A reprogrammable low power platform for closed-loop optogenetics," *IEEE Trans. Biomed. Eng.*, vol. 67, no. 11, pp. 3004–3015, 2020.

[20] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *J. Mach. Learn. Res.*, vol. 17, no. 83, pp. 1–5, 2016.

[21] R. Schwan, Y. Jiang, D. Kuhn, and C. N. Jones, "PIQP: A proximal interior-point quadratic programming solver," in *Proc. IEEE Conf. Decision Control (CDC)*, 2023.

[22] V. Tjeng, K. Y. Xiao, and R. Tedrake, "Evaluating robustness of neural networks with mixed integer programming," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[23] S. Mehrotra, "On the implementation of a primal-dual interior point method," *SIAM J. Optim.*, vol. 2, no. 4, pp. 575–601, 1992.

[24] S. Pougkakiotis and J. Gondzio, "An interior point–proximal method of multipliers for convex quadratic programming," *Comput. Optim. Appl.*, vol. 78, no. 2, pp. 307–351, 2021.

[25] A. Dvoretzky, J. Kiefer, and J. Wolfowitz, "Asymptotic minimax character of the sample distribution function and of the classical multinomial estimator," *Ann. Math. Statist.*, vol. 27, no. 3, pp. 642–669, 1956.

[26] P. Massart, "The tight constant in the Dvoretzky–Kiefer–Wolfowitz inequality," *Ann. Probab.*, vol. 18, no. 3, pp. 1269–1283, 1990.

[27] A. Maurer and M. Pontil, "Empirical Bernstein bounds and sample variance penalization," in *Proc. Conf. Learn. Theory (COLT)*. Omnipress, 2009, pp. 115–124.

[28] I. Gibbs and E. Candès, "Adaptive conformal inference under distribution shift," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 1660–1672, 2021.

[29] M. Contin and P. Martinelli, "Pharmacokinetics of levodopa," *J. Neurol.*, vol. 257, no. Suppl 2, pp. S253–S261, 2010.

[30] S. F. Lempka, S. Miocinovic, M. D. Johnson, J. L. Vitek, and C. C. McIntyre, "In vivo impedance spectroscopy of deep brain stimulation electrodes," *J. Neural Eng.*, vol. 6, no. 4, p. 046001, 2009.